
GUIDE PRATIQUE

Coder avec l'IA

De l'idée à une vraie
application livrée

Le journal complet d'un projet réel :
du cahier d'écopier de ma mère
à l'application en ligne.

Les 40 prompts qui ont servi.
Les ratés comme les bons.

Yedidya Kahire Gandura

Go · PostgreSQL · React — 2026

Licence personnelle

Cet exemplaire est enregistré au nom de :

```
Nom      · {{nom_acheteur}}  
Courriel · {{email_acheteur}}  
Achat    · {{date_achat}} – {{reference}}
```

Vous pouvez lire ce livre sur tous vos appareils et en imprimer un exemplaire pour votre usage personnel. Vous ne pouvez ni le partager, ni le revendre, ni le mettre en ligne, ni le distribuer sous quelque forme que ce soit.

Chaque exemplaire porte une empreinte unique liée à l'acheteur. Un fichier retrouvé en circulation est traçable jusqu'à son acquéreur d'origine.

Si ce livre vous a été transmis par quelqu'un d'autre et qu'il vous est utile, procurez-vous votre propre exemplaire. C'est ce qui permet à ce travail d'exister.

© 2026 Yedidya Kahire Gandura. Tous droits réservés.

Première édition – 2026. Goma, République démocratique du Congo.

Le code du projet compagnon est distribué séparément sous ses propres conditions.

PRÉFACE

Un cahier d'écolier

Ma mère élève des poulets à Goma. Deux ou trois lots en même temps, un millier de bêtes chacun. Pendant des années, tout ce qui entrait et tout ce qui sortait de ces poulaillers a été écrit à la main, dans des cahiers d'écolier à couverture cartonnée.

Une colonne pour la date. Une pour les poussins arrivés. Une pour les sacs de nourriture. Une pour les morts du jour. Une pour les ventes.

Ce système a tenu longtemps. Il tient encore aujourd'hui, chez énormément de gens, dans énormément de commerces — à deux rues d'ici comme à deux mille kilomètres.

Puis les cahiers s'allongent. On en remplit un, on en commence un autre. On cherche une dépense de la cinquième semaine et il faut remonter quarante pages à l'envers. Et parfois — c'est arrivé — un cahier se perd.

Ce jour-là, ce n'est pas du papier qui disparaît. C'est le coût réel d'un lot. C'est la réponse à la seule question qui compte à la fin : est-ce qu'on a gagné de l'argent, et combien.

Mes frères et moi avons lancé notre propre élevage, bien plus petit : deux cents poulets. Nos bêtes grandissent dans le même bâtiment que celles de notre mère. Nous lui achetons la nourriture et nous la remboursons après la vente.

Il faut donc suivre deux affaires distinctes qui partagent le même toit, la même nourriture, la même eau — et les mêmes pertes.

Essayez de tenir ça dans un cahier.

Je suis développeur. Ce problème se trouvait à quelques mètres de mon bureau, et pendant ce temps-là je construisais des listes de tâches et des clones d'applications connues, comme tout le monde.

Ce livre est le journal complet de l'application que j'ai fini par écrire pour ma famille. De la première question posée à ma mère jusqu'à la mise en ligne.

Ce n'est pas un livre sur l'intelligence artificielle, ni un recueil de prompts à recopier : les prompts vieillissent, et celui qui fonctionne aujourd'hui sera périmé dans un an.

C'est un livre sur une compétence qui, elle, ne vieillit pas : savoir reconnaître qu'une réponse d'une IA est insuffisante, et savoir dire pourquoi.

Vous verrez donc ici ce que les tutoriels ne montrent jamais. Les prompts tapés en premier, mal formulés. Le code qu'ils ont produit, faux. Le moment où j'ai compris qu'il l'était. Et le prompt qui a fini par fonctionner.

À la fin de ce livre, vous n'aurez pas « appris l'IA ». Vous aurez livré quelque chose : une application qu'une personne réelle utilise tous les jours. C'est ce qui manque à la plupart des développeurs qui cherchent leur premier client.

Je n'écris pas depuis la Silicon Valley, mais depuis Goma, en République démocratique du Congo. Ma première utilisatrice est ma mère. C'est peut-être ce qui rend ce livre utile : rien ici n'est théorique.

Yedidya Kahire Gandura
Goma, 2026

AVANT DE COMMENCER

À qui s'adresse ce livre

Ce livre s'adresse à une personne précise. Lisez les lignes qui suivent : vous saurez en deux minutes s'il a été écrit pour vous.

Il est fait pour vous si :

- Vous savez coder — au moins lire et modifier du code — mais vous n'avez jamais mis en ligne une application que quelqu'un d'autre utilise vraiment.
- Vous vous servez déjà d'une IA pour écrire du code, et vous obtenez souvent quelque chose qui fonctionne sans savoir si c'est juste.
- Votre portfolio est rempli de projets d'exercice et ne prouve pas que vous savez livrer.
- Vous cherchez votre premier client payant.

Il n'est pas fait pour vous si :

- **Vous n'avez jamais programmé.** Ce livre suppose que vous savez lire une fonction et lancer une commande dans un terminal. Si ce n'est pas encore votre cas, commencez plutôt par *Coder à partir de zéro* (à paraître), qui est le livre d'avant celui-ci. Revenez ensuite : tout ce qui suit vous sera accessible.
- **Vous cherchez un cours sur Go, React ou PostgreSQL.** Ces outils sont ici des moyens, jamais des sujets : je m'en sers sans les enseigner. Pour les apprendre pour eux-mêmes, *La pile complète* (à paraître) leur est entièrement consacré.
- **Vous voulez une liste de prompts à copier sans comprendre.** Vous les trouverez tous ici — l'annexe B les rassemble — mais jamais sans la raison pour laquelle ils fonctionnent. Cette raison est la seule chose qui ne se périme pas. Je n'écirai aucun livre qui vende des prompts nus.

Les titres signalés « à paraître » sont en cours d'écriture. Pour être prévenu de leur sortie :
{ {lien_liste_attente} }

Ce qu'il faut pour commencer

Il vous faut un ordinateur, un éditeur de code et l'accès à un assistant IA — n'importe lequel fera l'affaire. Rien d'autre ne s'achète.

Le seul prérequis qui compte ne s'achète pas non plus : un vrai problème, chez quelqu'un que vous connaissez. Le premier chapitre y est consacré.

Le projet qui sert de fil rouge

Ce livre suit un seul projet du début à la fin : une application de suivi d'élevage, écrite pour ma mère et mes frères. Arrivages, nourriture, mortalité, ventes — y compris à crédit — et le calcul de ce qu'un lot a réellement rapporté.

Plusieurs propriétaires partagent le même bâtiment. C'est ce détail qui rend les calculs bien moins simples qu'ils n'en ont l'air — et c'est là que l'IA se trompe le plus.

Vous n'avez pas besoin de vous intéresser aux poulets. Ce qui compte, c'est que chaque décision de ce livre a été prise devant un problème réel, avec une utilisatrice qui protestait quand ça ne marchait pas.

Ce que vous saurez faire à la fin

- Transformer le besoin flou d'une personne non technique en cahier des charges exploitable.
- Reconnaître qu'une réponse d'IA est insuffisante, et dire précisément ce qui manque.
- Écrire un prompt qui produit du code juste sur une règle métier que l'IA ne connaît pas.
- Modéliser une base de données qui résiste aux cas réels, pas seulement aux cas d'école.
- Gérer l'argent dans une application sans perdre une écriture ni fausser un arrondi.
- Déployer, sauvegarder et surveiller une application en production.
- Appuyer vos prompts sur des références reconnues, pour obtenir du code écrit comme l'écrirait un professionnel.
- Transformer cette application en proposition commerciale pour un premier client.

Huit promesses. Le reste du livre les paie une par une, dans l'ordre où vous en aurez besoin.

Sommaire

Préface — Un cahier d'écologiste 3

À qui s'adresse ce livre 6

PARTIE I

Avant la première ligne de code

1 Partir d'un vrai problème 12

2 Du besoin flou au cahier des charges 20

3 Choisir sa pile et son architecture 38

PARTIE II

Concevoir

4 Le design avant le code 48

5 Modéliser la base de données 64

PARTIE III

Construire

6 Le back-end, brique par brique 76

7 La logique métier que l'IA rate 89

8	Le front-end : un écran, c'est cinq états	100
---	---	-----

9	L'argent : suivre n'est pas encaisser	106
---	---	-----

PARTIE IV

Livrer

10	Tester ce que l'IA a écrit	112
----	----------------------------------	-----

11	Déployer et survivre à la production	117
----	--	-----

12	De l'application au client payant	122
----	---	-----

ANNEXES

À garder sous la main

A	Les douze diagnostics, réunis	128
---	-------------------------------------	-----

B	La bibliothèque de prompts	130
---	----------------------------------	-----

C	Les six briques d'un prompt qui marche	131
---	--	-----

D	La checklist de livraison	132
---	---------------------------------	-----

E	Le modèle de base de données complet	133
---	--	-----

F	Les références à donner à l'IA	134
---	--------------------------------------	-----

G	Glossaire	135
---	-----------------	-----

Partir d'un vrai problème

L'IA ne trouve pas les problèmes. Elle ne fait que les traiter.

Ouvrez le portfolio d'un développeur qui cherche son premier contrat. Vous y trouverez une application de tâches. Un clone de Twitter. Un site de météo. Un tableau de bord rempli de chiffres inventés.

Ces projets ont tous le même défaut : personne ne les utilise. Pas même celui qui les a écrits.

Ce n'est pas un problème de compétence. Le code est parfois très bon. C'est un problème de preuve.

Un client ne cherche pas quelqu'un qui sait écrire du code — l'IA écrit du code. Il cherche quelqu'un capable de transformer une situation confuse en logiciel qui tient debout. Un clone de Twitter ne prouve rien de tel : la situation était déjà claire, et quelqu'un d'autre l'avait résolue avant vous.

Quatre critères, et pas un de plus

Un problème mérite qu'on écrive une application s'il remplit quatre conditions. Toutes se vérifient en une journée, sans méthode compliquée et sans questionnaire.

- **Quelqu'un le vit sans vous.** La personne concernée n'est pas vous. Vous pouvez être utilisateur aussi, mais vous ne pouvez pas être le seul.
- **Il coûte quelque chose de mesurable.** Du temps perdu à chercher, de l'argent mal compté, une information qui disparaît. Si personne ne perd rien, personne ne changera ses habitudes pour vous.
- **Vous pouvez parler à cette personne cette semaine.** Pas dans un mois, pas par formulaire. Vous allez avoir cent questions à poser, et la plupart sont des questions bêtes qu'on ne pose qu'en confiance.
- **Elle a déjà bricolé une solution.** C'est le critère le plus important, et le moins évident. La section suivante lui est entièrement consacrée.

Ce que quelqu'un fait déjà à la main

Un problème réel laisse toujours une trace. Quelqu'un a déjà mis en place un système pour s'en sortir — un système qui fonctionne mal, mais qui fonctionne.

Cette trace prend des formes très différentes. Un cahier posé sur un comptoir. Un tableur rempli de cases coloriées à la main. Un groupe WhatsApp où l'on poste des photos de reçus. Un tableau au marqueur sur un mur.

Parfois, il n'y a rien d'écrit du tout. La trace est alors un geste répété : un appel passé chaque lundi pour savoir où en sont les livraisons, une même question posée toutes les semaines à la même personne parce qu'elle est seule à connaître la réponse.

La forme importe peu. Ce qui compte est ce qu'elle prouve : quelqu'un a jugé cette information assez importante pour en payer le prix — du temps, de la discipline, des ratures — tous les jours, pendant des mois.

On vend des méthodes entières pour valider une idée d'application. Ceci fait mieux, et gratuitement : ce n'est pas une intention déclarée, c'est un comportement déjà payé.

Ce que j'ai eu sous les yeux

Ma mère tenait ses cahiers à quelques mètres de mon bureau.

Je les voyais tous les jours. Je les ai vus s'empiler. Je l'ai vue chercher une dépense en remontant les pages à l'envers — et je n'y voyais qu'une contrainte normale de son métier.

Pendant ce temps, je construisais des applications de tâches.

Le déclic est venu un soir de règlement des comptes. On venait de clôturer un lot partagé — neuf cents bêtes pour ma mère, deux cents pour mes frères et moi. Le cahier était ouvert sur la table, rempli de colonnes qu'il fallait remonter sur deux mois. Ma mère a passé une demi-heure à recompter les dépenses pour séparer ce qui revenait à chacun. À la fin, le total ne tombait pas juste. Personne ne savait si l'erreur était une dépense oubliée ou un report mal recopié. On a fini par partager la différence à l'amiable, sans savoir qui avait tort.

Ce retard n'a rien d'exceptionnel. Nous cherchons des problèmes sur internet, dans des listes d'idées, dans les projets des autres — et pas devant nous.

Un problème que vous côtoyez depuis des années ne

Quatre pièges

- **Le problème inventé.** Vous l'avez déduit d'un raisonnement au lieu de l'observer. Le signe qui ne trompe pas : vous êtes incapable de nommer la personne qui le vit.
- **Le problème trop grand.** « Une application pour gérer toute l'exploitation. » Coupez jusqu'à ce qu'il ne reste qu'une seule question à laquelle l'application doit répondre. Vous élargirez après.
- **L'utilisateur injoignable.** Le problème est réel, mais la personne est loin et ne répondra pas à vos questions. Vous coderez à l'aveugle, et vous vous tromperez sur les détails qui comptent.
- **Le tableur qui marche déjà.** Certaines personnes ont un fichier Excel excellent, rapide, qu'elles maîtrisent mieux que vous ne maîtriserez jamais votre application. Passez votre chemin — ou trouvez ce que le tableur ne sait pas faire : plusieurs personnes qui saisissent en même temps, l'accès depuis un téléphone, le calcul refait à chaque écriture.

L'IA à cette étape

Ce bloc revient à chaque étape du livre. Il montre ce que j'ai demandé, ce que j'ai obtenu, pourquoi ce n'était pas bon, et le prompt qui a fini par fonctionner.

Ce chapitre-ci est le seul où ce bloc dit non.

CE QUE J'AI DEMANDÉ AU DÉBUT

donne moi 10 idées d'applications à construire pour mon portfolio, quelque chose d'original qui impressionne un recruteur

CE QUE J'AI OBTENU – ET POURQUOI CE N'ÉTAIT PAS BON

Dix idées correctes. Un suivi d'habitudes. Une application de covoiturage entre voisins. Un gestionnaire de recettes avec liste de courses automatique.

Rien n'était faux. Tout était inutilisable.

L'IA n'a accès à aucun cahier. Elle ne connaît personne. Elle ne peut produire que la moyenne de ce qui s'écrit sur internet — c'est-à-dire, très exactement, les projets qui remplissent déjà tous les portfolios.

Demander une idée à une IA, c'est demander l'idée que tout le monde va avoir en même temps que vous.

CE QU'ELLE SAIT FAIRE, EN REVANCHE

L'IA ne trouve pas le problème. Mais elle l'interroge très bien, à condition que ce soit vous qui l'apportiez.

Voici une situation réelle que j'observe : [décris-la en dix lignes, avec des chiffres].

Ne me propose aucune solution et ne me suggère aucune fonctionnalité.

Pose-moi les quinze questions qu'un analyste poserait à cette personne avant d'écrire la moindre ligne de code.

Classe-les de la plus gênante à la moins gênante.

Trois éléments font tout le travail : l'interdiction de proposer une solution, le nombre exact de questions, et l'ordre imposé. Sans eux, l'IA retombe sur son réflexe par défaut : proposer des fonctionnalités.

LA RÈGLE À RETENIR

L'IA ne trouve pas les problèmes, elle ne fait que les traiter. Le problème, c'est vous qui l'apportez — et vous l'apportez de la vie réelle, jamais d'une liste d'idées.

Avant de passer au chapitre 2

Ne lisez pas la suite tant que vous n'avez pas ces quatre choses. Le chapitre 2 part du principe qu'elles existent.

- **Un problème concret**, vécu par quelqu'un d'autre, que vous savez décrire en une phrase sans employer le mot « application ».
 - **Le nom de cette personne**, et un moyen de lui parler cette semaine.
 - **Sa trace** : ce qu'elle fait aujourd'hui pour s'en sortir. Cahier, tableur, appel du lundi, mémoire d'une seule personne — peu importe la forme, mais sachez la décrire.
 - **Une seule question**, écrite en une phrase, à laquelle sa méthode actuelle répond mal aujourd'hui.
-

Si vous n'avez pas encore votre problème, cherchez encore. C'est la seule étape de ce livre qu'aucun outil ne fera à votre place, et c'est celle qui décide de tout le reste.

Du besoin flou au cahier des charges

Personne ne sait décrire son besoin. Tout le monde sait montrer comment il fait.

Vous avez un problème et une personne. Il faut maintenant transformer ce qu'elle vit en quelque chose qui se construit.

L'erreur la plus courante à cette étape est de poser la question qui paraît la plus logique : « qu'est-ce que vous voulez ? »

Elle ne donne jamais rien d'exploitable. Non pas parce que les gens seraient confus, mais parce que traduire une activité en logiciel n'est pas leur métier. C'est le vôtre.

Vous obtiendrez l'une de ces trois réponses : la description de la méthode actuelle, une fonctionnalité aperçue dans une autre application, ou un souhait si large qu'il ne veut rien dire — « je veux tout voir d'un coup ».

Ne demandez pas, faites refaire

La seule méthode qui produit de la matière fiable tient en une phrase : demandez à la personne de faire devant vous ce qu'elle fait d'habitude, et taisez-vous.

Vous verrez ce qu'aucun entretien ne donne : l'ordre réel des gestes, les hésitations, les endroits où elle recompte, les moments où elle appelle quelqu'un pour vérifier, et les cases laissées vides depuis des mois parce qu'elles ne servent à rien.

Notez tout, y compris ce qui vous semble inutile. À ce stade, vous ne savez pas encore ce qui compte.

Trois questions produisent presque toute la valeur restante :

- « **Montrez-moi la dernière fois que ça s'est mal passé.** » Les exceptions représentent la moitié du travail d'un logiciel.
- « **Que faites-vous quand rien ne correspond ?** » Vous découvrirez les règles non écrites.
- « **Qui d'autre a besoin de cette information ?** » Vous découvrirez les utilisateurs que vous n'aviez pas vus.

Quatre listes suffisent

Oubliez les documents de quarante pages. Ce dont vous avez besoin tient en quatre listes, et chacune se remplit avec ce que vous venez d'observer.

- **Les acteurs.** Qui saisit, qui lit, qui décide. Distinguez ces rôles même s'il s'agit aujourd'hui d'une seule personne : c'est ce qui vous évitera de tout réécrire quand une deuxième arrivera.
- **Les objets.** Les choses que le métier nomme et compte. Pas des tables, pas des classes : les mots que la personne emploie devant vous.
- **Les règles.** Toutes les phrases qui commencent par « toujours », « jamais », « sauf si », « ça dépend ». Ce sont elles qui feront échouer l'IA au chapitre 7.
- **Les questions.** Celles auxquelles l'application devra répondre en moins de dix secondes.

Une application est une liste de questions

La quatrième liste est la plus importante, et c'est celle qu'on oublie.

Un logiciel de gestion répond vite à quelques questions qu'on posait lentement avant. Si vous ne savez pas les énoncer, vous ne savez pas ce que vous construisez — et vous livrerez un formulaire de saisie que personne n'ouvrira.

Dans mon cas, elles tenaient en six lignes : combien de bêtes sont vivantes aujourd'hui ; combien ce lot a coûté jusqu'ici ; qui **nous** doit de l'argent sur les lots déjà vendus ; à qui **nous** devons de l'argent sur le lot en cours ; combien je gagnerais par poulet si je vendais aujourd'hui ; que se passe-t-il si j'attends deux semaines de plus.

Regardez la troisième et la quatrième. « Qui doit de l'argent » ne veut rien dire : l'argent circule dans les deux sens, et sur des lots différents. Deux questions, deux écrans, deux réponses.

Une question vague produit un écran vague. Chaque fois qu'un mot lève une ambiguïté — *lequel, à qui, sur quelle période* — ajoutez-le : il vous économisera une refonte.

Le vocabulaire du métier n'est pas négociable

Si la personne dit « lot », votre application dit « lot ». Jamais « batch », jamais « groupe », jamais « unité de production ».

Ces mots-là deviendront le nom de vos tables au chapitre 5, le titre de vos écrans au chapitre 8, et les termes que vous emploierez le jour où vous formerez quelqu'un. Chaque traduction que vous vous autorisez ici, vous la paierez trois fois.

Attention aussi aux mots qui semblent évidents et ne le sont pas. Demandez systématiquement une définition, même quand vous croyez avoir compris — surtout quand vous croyez avoir compris.

Chez nous, une « sortie » ne désignait pas une vente. C'était toute bête qui quitte le bâtiment : vendue, morte, offerte ou consommée à la maison. J'ai mis trois semaines à m'en apercevoir, et mes premiers calculs de rentabilité étaient tous faux.

Ouvrez un glossaire dès aujourd'hui. Un mot par ligne, la définition dans les mots de la personne, pas dans les vôtres.

Les six briques d'un prompt qui fonctionne

Tout ce qui suit dans ce livre repose sur la même structure. Un prompt utile contient six éléments : retirez-en un, la qualité baisse ; retirez-en deux, vous obtenez du générique.

- 1 **Le rôle et le contexte.** Qui vous êtes, ce que vous construisez, pour qui.
 - 2 **La matière réelle.** Vos notes, les phrases exactes, vos chiffres. Jamais un résumé.
 - 3 **Les contraintes.** Ce qui est interdit. La brique la plus souvent oubliée.
 - 4 **Un exemple chiffré.** Un cas complet, avec de vrais nombres et le résultat attendu.
 - 5 **La référence.** La documentation ou l'ouvrage dont vous voulez qu'il suive la discipline.
 - 6 **Le format de sortie.** Ce que vous voulez recevoir, et sous quelle forme.
-

Les prompts ratés manquent presque toujours des briques 3 et 4.

L'IA à cette étape · 2.1 — Structurer

CE QUE J'AI DEMANDÉ AU DÉBUT

écris-moi un cahier des charges complet pour une application de gestion d'élevage de poulets

CE QUE J'AI OBTENU

Cahier des charges — Application de gestion d'élevage avicole

- 1. Objectif.** Permettre le pilotage complet d'un élevage, de l'acquisition des poussins à la commercialisation.
- 2. Utilisateurs.** Administrateur, employés de ferme, vétérinaire (accès consultation).
- 3. Modules fonctionnels.**
 - 3.1 Gestion des lots : création, suivi, historique
 - 3.2 Gestion des stocks : aliments, médicaments, alertes de seuil
 - 3.3 Suivi sanitaire : vaccinations, traitements, carnet vétérinaire
 - 3.4 Suivi de production : poids moyen, indice de consommation
 - 3.5 Gestion commerciale : clients, devis, commandes, factures
 - 3.6 Tableau de bord : taux de mortalité, rentabilité, KPI
 - 3.7 Multi-sites : gestion de plusieurs bâtiments

4. Exigences non fonctionnelles. Interface responsive, mode hors ligne, sauvegarde automatique, gestion des rôles et permissions, export PDF et Excel.

POURQUOI CE N'ÉTAIT PAS BON

Le document est propre. Il est bien découpé. On a envie d'y croire.

Et il ne décrit pas mon élevage.

Sept modules sont proposés ; je n'en avais demandé aucun. Le carnet vétérinaire, l'indice de consommation, les devis, le multi-sites : rien de tout cela n'existe chez nous, et rien de tout cela ne serait jamais ouvert.

Surtout, ce cahier des charges ignore les trois choses qui rendaient mon cas difficile : des poulets de propriétaires différents qui grandissent dans le même bâtiment, une valeur par tête qui change à chaque dépense, et des ventes à crédit dont l'argent reste chez ma mère jusqu'au règlement.

L'IA n'avait rien pour deviner tout ça. Je ne lui avais rien donné.

LE PROMPT QUI FONCTIONNE

Tu es analyste. Je construis une application pour une seule exploitation familiale, utilisée par cinq personnes.

Voici mes notes brutes : un arrivage physique peut contenir les poulets de plusieurs propriétaires (1200 = 1000 à ma mère + 200 à mes frères et moi) et ils doivent rester séparés. On note chaque décès, chaque dépense (eau, électricité, transport, vaccins) et chaque prélèvement de nourriture. La valeur d'un poulet = (achat + dépenses) ÷ poulets restants ; elle change à chaque dépense. Les ventes se font au comptant ou à crédit. L'argent des ventes reste chez ma mère jusqu'au règlement des comptes.

N'invente aucune fonctionnalité. Quand une information manque, écris MANQUE et pose la question.

Cas à couvrir : 1000 poulets à 1 \$, 2000 \$ de dépenses en semaine 5, 200 morts.

Formule chaque règle au format Given / When / Then (Gherkin).

Rends-moi quatre listes : acteurs, objets, règles, questions.

CE QUE J'AI OBTENU CETTE FOIS

ACTEURS

- La mère : propriétaire principal, détient l'argent des ventes jusqu'au règlement.
- Les quatre frères : copropriétaires d'un lot plus petit.
- MANQUE : qui a le droit de corriger une saisie erronée ?

OBJETS

- Arrivage : date, nombre, prix unitaire, propriétaires.
- Lot : part d'un arrivage appartenant à un propriétaire.
- Dépense, Décès, Prélèvement de stock, Vente.
- MANQUE : « lot » désigne-t-il l'arrivage physique ou la part d'un propriétaire ?

RÈGLES

- Given 1000 poulets à 1 \$ et 2000 \$ de dépenses, When 200 sont morts, Then la valeur par poulet vaut $3000 \div 800 = 3,75$ \$.
- Given un arrivage partagé, When 40 poulets meurent, Then MANQUE : comment répartir les morts entre propriétaires ?

QUESTIONS

- Combien de bêtes vivantes, par propriétaire, aujourd'hui ?
- Combien ce lot a-t-il coûté jusqu'ici ?
- Qui doit encore de l'argent sur une vente à crédit ?
- MANQUE : une dépense sur un arrivage partagé se répartit-elle au prorata du nombre de poulets ?

Le plus utile n'est pas ce qu'elle a rédigé. Ce sont les MANQUE.

Celui sur la répartition des morts entre propriétaires est la question la plus difficile de tout le projet, et je ne me l'étais jamais posée en trois ans d'observation.

Les six briques, dans ce prompt

Relisez-le : rôle et contexte, notes brutes, interdiction d'inventer, exemple chiffré, référence Gherkin, format de sortie.

LA RÈGLE À RETENIR

L'IA structure ce que vous lui donnez. Tout ce qu'elle ajoute d'elle-même est du remplissage — plausible, bien écrit, et faux.

L'IA à cette étape · 2.2 — Le contradicteur

Vous ne verrez pas les trous de votre cahier des charges. C'est vous qui l'avez écrit : vous savez ce que vous vouliez dire, donc vous le relisez comme s'il était clair.

Il faut quelqu'un pour l'attaquer. L'IA fait très bien ce travail-là, parce qu'elle n'a aucune raison de vous ménager.

LE PROMPT

Voici mon cahier des charges : [coller].

Ne le valide pas. Ne le complimente pas.

Liste vingt situations réelles dans lesquelles il ne dit pas quoi faire. Pour chacune, indique quelle règle manque.

Classe de la plus probable à la moins probable.

CE QUE ÇA M'A COÛTÉ

Quatre de ses réponses m'ont obligé à tout reprendre. Aucune n'était compliquée : ce sont des choses qui arrivent tous les mois.

- Une bête meurt après la vente, mais avant que le client vienne la chercher. Qui perd ?
- Un client paie la moitié. Le lot est-il vendu ou pas ?

- On vend la bête d'un propriétaire à la place de celle d'un autre. Comment le rattraper au moment des comptes ?
 - Une dépense arrive après la clôture du lot. Faut-il rouvrir le lot, ou la reporter sur le suivant ?
-

Aucune de ces quatre situations n'était dans mon cahier des charges. Toutes les quatre s'étaient déjà produites dans la vraie vie.

Ce que j'ai répondu

Je n'ai pas pu répondre seul. J'ai dû retourner voir ma mère avec la liste. C'est la première leçon du contradicteur : il ne vous donne pas de réponses, il vous renvoie sur le terrain.

Une bête meurt avant la remise

La perte est pour nous. On ne remet pas une bête morte à un client. Tant que l'animal n'a pas quitté le bâtiment, il nous appartient.

Un client paie la moitié

La vente existe, et le crédit aussi. Cette réponse a fait tomber ma première maquette, où j'avais prévu une case « payée / non payée ».

Une vente enregistre quatre nombres : combien de bêtes, combien est dû, combien est payé, combien reste. Et le reste se règle en plusieurs fois, au rythme du recouvrement.

Symétriquement, nous achetons nous-mêmes à crédit — nourriture, médicaments. L'application doit suivre les dettes dans les deux sens.

La question qui n'avait pas de réponse

La troisième supposait qu'on puisse vendre la bête d'un propriétaire à la place d'un autre. Ma mère n'a pas compris la question.

Chez nous, les bêtes n'appartiennent à personne individuellement. Un arrivage est un pot commun, et chaque propriétaire en détient une part.

- Mille bêtes : neuf cents à un propriétaire, cent à l'autre.
- Dix meurent : le premier en perd neuf, le second une. Personne ne va voir lesquelles.
- On vend sans regarder à qui elles sont : quarante à six dollars aujourd'hui, quarante à sept dollars demain.
- À la fin, on calcule le prix moyen. S'il s'établit à six dollars, le second propriétaire est payé pour ses quatre-vingt-dix-neuf bêtes restantes, et le premier pour ses huit cent quatre-vingt-onze.

Ce n'était donc pas un trou dans mon cahier des charges. C'était une erreur dans ma tête : j'avais modélisé des poulets, alors que le métier gère des parts.

CE QUE ÇA M'A APPRIS

Le contradicteur ne sert pas seulement à trouver les oublis. Il révèle surtout les questions mal posées — c'est-à-dire vos hypothèses.

La dépense en retard

Celle-là, ma mère a tranché sans hésiter : on ne rouvre jamais un lot clôturé. La dépense est reportée sur le lot suivant — à condition que ce soient les mêmes propriétaires qui y travaillent ensemble.

Relisez la fin de la phrase. C'est une réponse qui pose une question : et si le lot suivant n'a pas les mêmes propriétaires ? Je l'ai ajoutée à ma liste. Chaque règle obtenue en fabrique une autre, et c'est le signe qu'on avance.

Ce qui reste ouvert

Un point n'est toujours pas tranché, et je l'ai laissé écrit noir sur blanc dans le cahier des charges.

La répartition au prorata produit des fractions. Sept morts sur un partage de neuf cents et cent, cela donne 6,3 et 0,7. Personne ne perd sept dixièmes de poulet.

Il faut donc une règle d'arrondi. Et quelle que soit celle qu'on choisit, c'est toujours le même propriétaire qui sera lésé — celui qui détient la plus petite part.

Ce genre de détail ne fait pas planter une application. Il fait bien pire : il fait qu'une famille cesse de s'en servir, sans jamais vous dire pourquoi.

Un cahier des charges honnête contient des trous marqués. C'est celui qui n'en contient aucun qui doit vous inquiéter.

LA RÈGLE À RETENIR

Demandez-lui de casser, jamais de valider. Une IA à qui vous demandez si votre travail est bon vous répondra qu'il est bon.

Avant de passer au chapitre 3

- **Vos notes brutes**, avec les phrases exactes de la personne et au moins cinq chiffres réels.
- **Les quatre listes** : acteurs, objets, règles, questions.
- **Le glossaire du métier** : chaque mot que la personne emploie, avec sa définition dans ses mots à elle.
- **La liste des trous** que le contradicteur a trouvés, et ce que vous avez décidé pour chacun.

Le chapitre 3 choisit les outils. Il est court, et volontairement moins important que celui-ci : une pile technique, on peut en changer. Un cahier des charges faux, on le paie pendant des mois.

Choisir sa pile et son architecture

Un projet livré avec des outils moyens bat un projet abandonné avec les outils parfaits.

Ce chapitre est le plus court du livre, et c'est volontaire.

Le choix des outils est le sujet sur lequel les développeurs passent le plus de temps en prenant le moins de risques. On peut comparer des langages pendant trois semaines sans écrire une ligne. C'est confortable, ça ressemble à du travail, et ça ne livre rien.

Une pile technique se remplace. Un cahier des charges faux se paie pendant des mois. Le travail difficile, vous venez de le faire.

Le seul critère qui compte au départ

Choisissez ce que vous savez déjà.

Pas ce qui est le plus rapide, pas ce qui a la plus belle documentation, pas ce dont on parle le plus cette année. Ce que vous savez déjà.

La raison est simple. Vous allez rencontrer des dizaines de problèmes qui n'ont rien à voir avec la technologie : des règles métier tordues, un utilisateur qui n'ouvre jamais l'application, une connexion qui tombe au mauvais moment. Si vous apprenez un langage en même temps, vous ne finirez pas.

Il n'y a qu'une exception : quand le problème impose lui-même une contrainte que vos outils habituels ne savent pas satisfaire. Alors, et alors seulement, on change d'outil.

Listez les contraintes avant les technologies

Avant de citer le moindre nom d'outil, écrivez ce que la situation impose. Cette liste-là n'est pas négociable, contrairement aux outils.

- **Qui saisit, et sur quoi.** Chez nous : cinq personnes, sur des téléphones d'entrée de gamme.
 - **Dans quelles conditions.** Une connexion qui coupe plusieurs fois par jour.
 - **Ce que l'application calcule.** De l'argent, et des répartitions au prorata. Donc des chiffres justes au centime, et des écritures qui ne se perdent jamais à moitié.
 - **Qui maintiendra.** Moi seul, le soir.
-

Ces quatre lignes éliminent plus d'options que n'importe quel comparatif de langages.

Ce que j'ai choisi, et la vraie raison

- **Go pour l'API.** La vraie raison : je le connais. La bonne raison, trouvée après coup : un seul fichier exécutable, facile à déployer sur un petit serveur.
- **PostgreSQL pour la base.** Ici le problème impose le choix : de l'argent, des répartitions au prorata, plusieurs personnes qui écrivent en même temps. Il faut des transactions et des nombres décimaux exacts.
- **React pour l'interface.** C'est le plus discutable des trois. J'y reviens au chapitre 8.

Remarquez l'ordre des phrases pour Go : la vraie raison avant la bonne. Faites cet exercice à chaque choix technique — auriez-vous pris la même décision en ne connaissant rien ?

L'IA à cette étape · 3.1 — L'argumentaire contre

CE QUE J'AI DEMANDÉ AU DÉBUT

quelle est la meilleure stack pour développer une application de gestion en 2026 ?

CE QUE J'AI OBTENU

Pour une application de gestion moderne, la pile la plus recommandée est :

Front-end — Next.js avec TypeScript et Tailwind CSS

Back-end — Node.js ou NestJS

Base de données — PostgreSQL avec Prisma

Authentification — Clerk ou Auth0

Hébergement — Vercel

Cette combinaison offre un excellent équilibre entre rapidité de développement, maturité de l'écosystème et scalabilité.

Cette réponse est typique de la période 2023-2025. Si vous lisez ce livre plus tard, l'IA vous proposera d'autres noms. La leçon ne changera pas : elle répond ce qui est populaire, pas ce qui vous convient.

POURQUOI CE N'ÉTAIT PAS BON

Cette réponse n'est pas fausse. Elle est simplement sans rapport avec moi.

Aucune des quatre contraintes de la page 40 n'y figure. Ni les téléphones d'entrée de gamme, ni la connexion qui coupe, ni le fait que je serai seul à maintenir ce code pendant des années.

Le mot « scalabilité » apparaîtrait pour une application que cinq personnes utiliseront.

Une IA à qui l'on demande le meilleur choix répond la moyenne de ce qui s'écrit sur internet : ce qui est populaire, pas ce qui vous convient. Et elle ne vous contredira jamais, parce que vous ne lui avez rien donné à contredire.

LE PROMPT QUI FONCTIONNE

J'ai déjà choisi : Go pour l'API, PostgreSQL, React pour l'interface.

Contexte : cinq utilisateurs, téléphones d'entrée de gamme, connexion qui coupe plusieurs fois par jour, calculs d'argent avec répartitions au prorata, un seul développeur qui maintient le soir.

Ne valide pas mon choix. Ne propose aucune alternative.

Donne-moi les cinq situations concrètes où ce choix me coûtera cher. Pour chacune, indique à quel moment du projet la facture arrivera.

CE QUE J'AI OBTENU

1. La connexion instable. Aucun de ces trois outils ne règle la synchronisation hors ligne. Vous devrez la construire vous-même. Facture : aux premiers essais sur le terrain.

2. React sur téléphone d'entrée de gamme. Le paquet JavaScript sera lourd au premier chargement, sur des appareils lents et une connexion faible. Facture : à la première démonstration.

3. Les montants en Go. Le type flottant trahit sur les calculs d'argent. Il faut des entiers en centimes ou une bibliothèque décimale, décidé avant la première table. Facture : très lourde si découvert après.

4. Le CRUD en Go est verbeux. Beaucoup de code répétitif pour peu de logique. Facture : étalée, en fatigue.

5. Vous serez seul. Trouver de l'aide sur Go autour de vous est plus difficile que sur PHP ou JavaScript. Facture : le jour où vous bloquez.

Trois de ces cinq points étaient de vraies découvertes.

Le troisième m'a fait changer une décision avant même d'écrire la première table. Vous le retrouverez au chapitre 9.

LA RÈGLE À RETENIR

Ne demandez jamais à une IA ce qu'il faut choisir. Dites-lui ce que vous avez choisi, et demandez-lui de le démolir.

L'architecture tient en trois boîtes

Une base de données, une interface, et une API entre les deux. Résistez à tout le reste : pas de microservices, pas de file de messages, pas de cache.

La règle est celle du chapitre 2, appliquée aux machines : l'architecture la plus simple qui réponde aux questions que vous avez écrites.

Si vous ne pouvez pas justifier une boîte par une de ces questions, elle n'existe pas. Vous l'ajouterez le jour où elle manquera vraiment, et ce jour-là vous saurez pourquoi.

Avant de passer au chapitre 4

- **Vos outils écrits noir sur blanc**, avec pour chacun la vraie raison, pas celle que vous diriez en entretien.
 - **Vos quatre contraintes** : qui saisit et sur quoi, dans quelles conditions, ce qui est calculé, qui maintiendra.
 - **Les cinq situations** où votre choix vous coûtera cher, avec le moment de la facture.
 - **Les décisions** que cette liste vous a déjà fait changer.
-

Le chapitre 4 dessine l'application avant de l'écrire. C'est celui où l'on est le plus tenté de sauter directement au code — et celui où ça coûte le plus cher.

Le design avant le code

Le papier rend les mauvaises idées gratuites. Le code les rend chères.

Vous savez ce que l'application doit faire. La tentation, maintenant, est d'ouvrir l'éditeur.

Ne le faites pas encore. Une mauvaise idée dessinée sur une feuille se corrige en dix secondes. La même idée codée pendant trois soirées se défend, parce qu'on a mal à l'abandonner.

Ce chapitre ne parle pas d'esthétique. Il parle de décider ce que l'utilisateur verra en premier, et de ce qu'il pourra faire sans réfléchir.

Commencez par le geste le plus fréquent

Ne dessinez pas un écran d'accueil. Cherchez d'abord le geste que l'utilisateur répétera tous les jours, et faites-en votre page d'accueil.

Presque tout le monde se trompe ici, et de la même façon : on met en accueil un résumé complet de la situation. C'est ce qui impressionne le plus quand on montre l'application, et c'est ce qui sert le moins quand on s'en sert.

Chez nous, le geste quotidien est le comptage des bêtes mortes. Il a lieu le matin, tous les matins, qu'il y ait quelque chose à signaler ou non.

L'écran d'accueil de l'application, c'est donc ça. Pas un tableau de bord : un compteur et un bouton.

Le contexte décide de la forme

Un écran ne se conçoit pas devant un grand moniteur, au calme. Il se conçoit dans la situation où il sera réellement utilisé.

Écrivez cette situation avant de dessiner quoi que ce soit : où se tient la personne, ce qu'elle a dans les mains, quelle lumière, quel appareil, quel réseau.

Dans notre cas : debout dans le bâtiment, tôt le matin, une seule main libre, un téléphone d'entrée de gamme, et souvent pas de réseau.

Chacune de ces cinq conditions élimine des choix. Une main libre interdit les gestes à deux doigts. Pas de réseau interdit le message « échec de l'enregistrement ». L'appareil lent interdit les animations. Vous n'avez pas encore choisi une couleur que la moitié des décisions sont déjà prises.

L'écran qu'on n'ouvre jamais

Voici la partie que j'ai failli rater complètement.

Une application de gestion ne sert à rien si personne ne pense à l'ouvrir. Or personne ne pense à ouvrir une application de gestion. On l'ouvre quand on a un problème, c'est-à-dire trop tard.

Chez nous, les deux premières semaines d'un lot demandent une surveillance du chauffage toutes les deux heures, jour et nuit. Un oubli à trois heures du matin peut coûter des dizaines de poussins.

La fonction la plus utile de l'application n'est donc pas un écran. C'est une notification.

Listez, dès la conception, tout ce que l'application doit rappeler — et à quelle heure. Ce sont des fonctionnalités à part entière, pas un détail à ajouter plus tard.

Dessinez sur papier, puis taisez-vous

N'ouvrez pas d'outil de maquette. Prenez une feuille et un stylo, dessinez trois écrans, photographiez-les.

Le papier a une vertu que rien ne remplace : il est visiblement provisoire. Personne n'ose critiquer une maquette qui a l'air finie ; tout le monde ose corriger un dessin au stylo.

Montrez-le à la personne concernée et appliquez la règle du chapitre 2 : demandez-lui de faire, et taisez-vous. Ne dites pas « ici tu appuies ». Regardez où son doigt se pose tout seul.

Notez ses hésitations, pas ses compliments. Une hésitation d'une seconde sur le papier deviendra un appel téléphonique une fois l'application installée.

L'IA à cette étape · 4.1 — Décrire l'usage

Le prompt qui suit n'a rien de naïf. Il donne le contexte, les fonctionnalités, les utilisateurs, et il demande explicitement de la simplicité. C'est le prompt qu'écrirait un développeur sérieux.

CE QUE J'AI DEMANDÉ AU DÉBUT

Je développe une application mobile de suivi d'élevage de poulets pour une exploitation familiale.

Fonctionnalités : suivi des lots, enregistrement des mortalités, des dépenses et des ventes, calcul de la rentabilité par lot.

Utilisateurs : cinq personnes, dont la propriétaire, peu à l'aise avec le numérique.

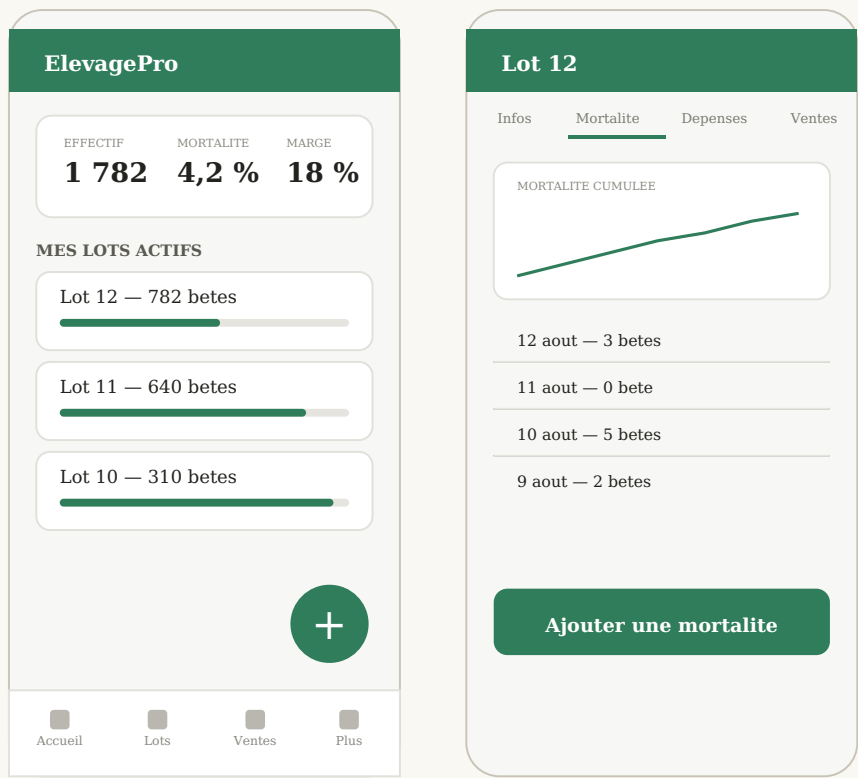
Propose-moi une interface mobile simple et claire, écran par écran.

CE QUE J'AI OBTENU

Écran d'accueil — en-tête au nom de l'exploitation, carte de synthèse (effectif, mortalité, marge), liste des lots actifs avec barre de progression, bouton flottant « + », barre d'onglets : Accueil, Lots, Ventes, Plus.

Écran détail du lot — quatre onglets : Informations, Mortalité, Dépenses, Ventes. Chacun affiche l'historique et un bouton d'ajout.

LA PROPOSITION, DESSINÉE TELLE QUELLE



Rien n'est laid. Rien n'est absurde. C'est exactement ce qui rend cette proposition difficile à refuser.

POURQUOI CE N'ÉTAIT PAS BON

Comptez maintenant les touchers du geste du matin.

Accueil, puis Lot 12, puis l'onglet Mortalité, puis « Ajouter une mortalité », puis saisir, puis valider. **Six touchers** — tous les jours, debout, une main occupée, pour un nombre qui vaut zéro la plupart du temps.

L'accueil affiche une marge et un taux de mortalité à quelqu'un qui, tôt le matin, veut seulement dire combien de bêtes sont mortes.

Et nulle part il n'est dit ce qui se passe sans réseau.

Ce prompt était bon selon tous les critères habituels. Il lui manquait la seule chose qui décide ici : la situation. J'avais décrit un logiciel, pas un matin.

LE PROMPT QUI FONCTIONNE – LES SIX BRIQUES

[1] Tu es concepteur d'interfaces. Je construis une application de suivi d'élevage utilisée par cinq personnes d'une même famille.

[2] Situation réelle de la saisie la plus fréquente : debout dans le bâtiment, tôt le matin, une seule main libre, téléphone Android d'entrée de gamme, souvent sans réseau. Le geste : compter les bêtes mortes et l'enregistrer. Il a lieu tous les jours, même quand le nombre est zéro.

[3] Interdits : barre latérale, effet de survol, animation, graphique tant qu'il n'y a pas six mois d'historique, message d'erreur en l'absence de réseau.

[4] Cas à couvrir : lot de 1000 bêtes, jour 34, 782 vivantes, 7 mortes ce matin, aucun réseau.

[5] Respecte les WCAG 2.2 : contraste, et cibles tactiles d'au moins 48 px.

[6] Rends-moi, écran par écran : ce qu'on voit, ce qu'on peut faire, le nombre de touchers, ce qui se passe sans réseau. Ensuite seulement, propose une direction visuelle sobre et soignée : une application utilisable n'a aucune raison d'être laide.

CE QUE J'AI OBTENU CETTE FOIS

ÉCRAN 1 — Aujourd'hui. Le nombre de bêtes vivantes en très grand, la date, l'état de connexion. Un bouton principal pleine largeur. Sans réseau : le dernier chiffre connu, daté, sans message d'erreur.

ÉCRAN 2 — Saisie. Un champ, un pavé numérique déjà ouvert. Deux touchers pour un nombre à un chiffre. Sans réseau : saisie acceptée, marquée en attente.

ÉCRAN 3 — Le lot. Répartition entre propriétaires, valeur par bête, dépensé et encaissé, derniers mouvements.

DIRECTION VISUELLE. Fond chaud plutôt que blanc pur, un seul accent, hiérarchie portée par la taille et non par la couleur, cartes aux angles doux.

MANQUE : si elle saisit deux fois dans la même journée, faut-il additionner ou remplacer ?

LES DEUX ÉCRANS DU GESTE QUOTIDIEN

Mercredi 26 aout ● hors ligne

782

betes vivantes

LOT 12 · JOUR 34
valeur par bete **3,75 \$**

Enregistrer des morts

Autre saisie

Synchronise hier a 18 h

Morts de ce matin

7

1 2 3
4 5 6
7 8 9
0 effacer

Valider

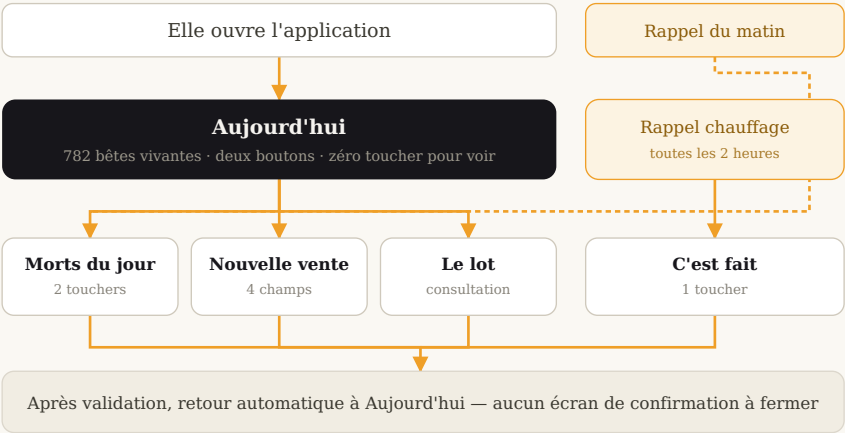
Deux touches au lieu de six. Le chiffre qui compte occupe le quart de l'écran.

LE MÊME LANGAGE, SUR UN ÉCRAN RICHE



Simplifier le geste quotidien n'oblige à appauvrir aucun autre écran.

COMMENT ON PASSE D'UN ÉCRAN À L'AUTRE



En pointillé : le rappel du matin ouvre directement la saisie, sans passer par l'accueil.

La carte complète. Cinq écrans, trois entrées, un seul chemin de retour.

Quatre règles de circulation

Des écrans dessinés séparément ne font pas un design. Ce qui fait le design, c'est le chemin entre eux — et c'est là que la plupart des applications de gestion se perdent.

- **Deux niveaux, jamais trois.** Depuis l'accueil, toute action est à un seul toucher. Si vous avez besoin d'un troisième niveau, c'est que l'accueil montre les mauvaises choses.
- **Retour automatique après validation.** Pas d'écran « Enregistré avec succès » à refermer. La personne est déjà repartie compter.
- **Un rappel ouvre l'action, jamais l'accueil.** C'est la règle la plus souvent ratée. Une notification qui dépose l'utilisateur sur la page d'accueil lui redemande le travail qu'elle venait justement de lui épargner.
- **Aucun menu caché.** Ni tiroir latéral, ni barre d'onglets. Ce qui n'est pas visible sur l'accueil n'existe pas pour quelqu'un qui utilise l'application debout, une main occupée.

Dessinez cette carte avant d'écrire le premier écran. Elle tient sur une feuille, et c'est elle qui vous dira si votre accueil est le bon.

Fonctionnel ne veut pas dire pauvre

Il existe une erreur symétrique à celle du tableau de bord, et elle coûte aussi cher : croire qu'une application destinée à une personne peu à l'aise avec le numérique doit être laide.

Personne n'ouvre volontiers un outil qui ressemble à un formulaire administratif. Le soin visuel n'est pas de la décoration : c'est ce qui donne envie de revenir, et l'envie de revenir décide de la survie de votre application.

Simplement, la beauté vient après les gestes, jamais avant. C'est pour cela que le prompt la demande en dernier, à la brique 6.

Deux écrans. Deux touchers. Et une question que je ne m'étais pas posée — celle du bas, qui n'a rien d'un détail d'interface : c'est une règle métier déguisée en problème de saisie.

LA RÈGLE À RETENIR

Décrivez l'utilisateur et la situation d'usage, jamais l'esthétique. Une IA à qui vous demandez du beau vous rendra la moyenne du web ; une IA à qui vous décrivez un matin dans un poulailler vous rendra deux écrans.

Avant de passer au chapitre 5

- **Le geste quotidien**, identifié et devenu votre écran d'accueil.
- **Les cinq conditions réelles** : lieu, mains, lumière, appareil, réseau.
- **La liste des rappels**, avec leurs horaires.
- **Trois écrans dessinés au stylo**, montrés à la personne concernée, avec ses hésitations notées.

Modéliser la base de données

Un écran se refait en une soirée. Un schéma faux se paie tant que l'application vit.

C'est le chapitre le plus important de la partie II, et celui où l'IA se trompe le plus discrètement.

Elle produira un schéma correct, normalisé, avec les bonnes clés étrangères. Rien à redire sur la forme. Et il sera incapable de répondre à la moitié de vos questions.

Parce qu'un schéma ne se juge pas à sa propreté. Il se juge à ce qu'on peut lui demander dans six mois.

Enregistrez les mouvements, pas les états

Voici la règle qui décide de tout le reste, et elle tient en une phrase : ne stockez jamais un chiffre que vous pouvez recalculer.

Un effectif, un solde, un total, un statut : ce sont des résultats. Si vous les rangez dans une colonne et que vous les mettez à jour, vous détruisez l'information à chaque écriture.

Une colonne *nombre_actuel* qui passe de 782 à 779 vous dit qu'il reste 779 bêtes. Elle ne vous dira jamais combien il en restait la semaine dernière, ni qui a fait la modification, ni pourquoi.

Enregistrez plutôt les événements : trois bêtes mortes, ce jour-là, saisies par cette personne. L'effectif se calcule. Et le jour où quelqu'un conteste un chiffre, vous avez la trace.

L'IA à cette étape · 5.1 — Le schéma

Le prompt de départ contient le cahier des charges résumé. Il est sérieux, il nomme les entités, il précise la base de données visée.

CE QUE J'AI DEMANDÉ AU DÉBUT

Conçois le schéma PostgreSQL d'une application de suivi d'élevage de poulets.

Entités : lots de poulets avec date d'arrivée et prix d'achat, mortalités quotidiennes, dépenses rattachées à un lot, ventes à des clients au comptant ou à crédit.

Donne-moi les tables, les colonnes, les types et les clés étrangères.

CE QUE J'AI OBTENU

Six tables, normalisées, avec index sur les clés étrangères.

fermes — l'exploitation.

lots — date d'arrivée, effectif initial, *effectif actuel*, prix unitaire, statut (actif / clos).

mortalites — date et nombre, rattachées au lot. Un déclencheur met à jour l'effectif actuel du lot.

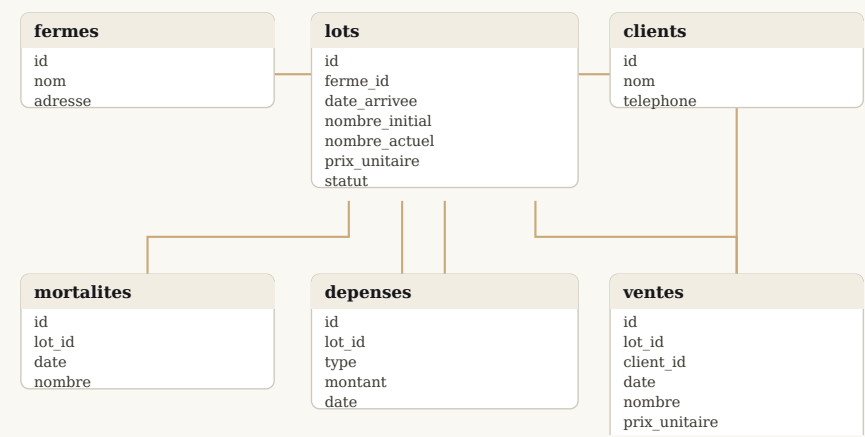
depenses — type, montant en NUMERIC(10,2), date.

ventes — client, nombre de bêtes, prix unitaire, et un booléen *paye*.

clients — nom et téléphone.

Le tout accompagné d'une remarque : « ce schéma suit les formes normales et sera performant en lecture. »

LE SCHÉMA COMPLET, TEL QUE PROPOSÉ



Six tables, rien à redire sur la forme. C'est ce qui rend ce schéma dangereux.

POURQUOI CE N'ÉTAIT PAS BON

Quatre défauts, dont trois sont invisibles tant qu'on ne pose pas les bonnes questions.

- **Les propriétaires n'existent pas.** Le schéma suppose qu'un lot appartient à une ferme. Chez nous, un arrivage appartient à plusieurs personnes en parts. Toute la comptabilité est impossible.
- ***effectif_actuel* est une colonne modifiable.** Chaque mortalité l'écrase. L'histoire disparaît à chaque écriture.
- ***paye* est un booléen.** Un client qui règle la moitié n'a pas de place dans ce schéma. C'est exactement la question du chapitre 2.
- **Les montants sont en NUMERIC.** Correct en soi, mais rien n'empêche l'application de les manipuler en flottant. C'est la facture n° 3 annoncée au chapitre 3.

LE PROMPT QUI FONCTIONNE – LES SIX BRIQUES

[1] Tu es concepteur de bases de données. PostgreSQL. Application de suivi d'élevage, exploitation familiale, cinq utilisateurs.

[2] Un arrivage physique appartient à plusieurs propriétaires en parts. Les bêtes ne sont jamais individuelles. Mortalité et dépenses se répartissent au prorata des parts. Les ventes se font sans distinguer les propriétaires ; le prix varie d'une vente à l'autre. Au règlement, chacun est payé selon le prix moyen. Un client peut payer en plusieurs tranches. L'exploitation elle-même achète à crédit.

[3] Aucune colonne ne doit contenir un chiffre recalculable : ni effectif courant, ni solde, ni total. Aucune correction par UPDATE. Tous les montants en entiers de centimes.

[4] Cas à couvrir : arrivage de 1000 bêtes à 1 \$, réparti 900 / 100 ; 2000 \$ de dépenses ; 200 mortes ; vente de 40 bêtes à 6 \$ dont 120 \$ payés.

[5] Suis la documentation PostgreSQL sur les contraintes CHECK et les types ENUM.

[6] Rends-moi les tables avec types et contraintes, puis la liste des vues calculées.

CE QUE J'AI OBTENU CETTE FOIS

Huit tables, et surtout deux idées que je n'avais pas.

parts — la table qui manquait. Un arrivage, un propriétaire, un effectif initial. C'est elle qui rend le prorata calculable.

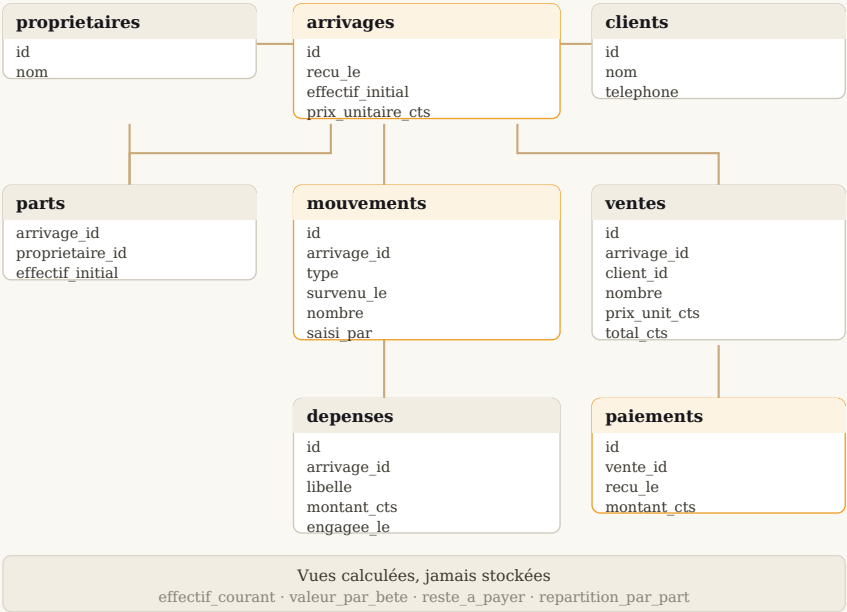
mouvements — un journal unique pour tout ce qui fait sortir une bête : mortalité, vente, don, consommation. Aucune colonne modifiable ; une erreur se corrige par un mouvement inverse.

paiements — une ligne par tranche reçue, rattachée à une vente. Le booléen *paye* disparaît : le reste dû devient une soustraction.

Et quatre vues, jamais stockées : effectif courant, valeur par bête, reste à payer, répartition par part.

MANQUE : quelle règle d'arrondi appliquer quand le prorata donne des fractions de bête ?

LE SCHÉMA COMPLET, APRÈS CORRECTION



En ambre, les trois tables qui n'existaient pas dans la première version.

Le journal des mouvements

Voici la table qui porte toute la règle. Notez ce qu'elle n'a pas : aucune colonne qu'on puisse modifier.

```
CREATE TYPE mouvement_type AS ENUM (  
    'mortalite', 'vente', 'don', 'consommation');  
  
CREATE TABLE mouvements (  
    id          BIGSERIAL PRIMARY KEY,  
    arrivage_id BIGINT NOT NULL REFERENCES arrivages(id),  
    type        mouvement_type NOT NULL,  
    survenu_le  DATE NOT NULL,  
    nombre      INTEGER NOT NULL CHECK (nombre > 0),  
    saisi_par   BIGINT NOT NULL REFERENCES utilisateurs(id),  
    cree_le     TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
-- Une erreur de saisie se corrige par un mouvement  
-- inverse, jamais par un UPDATE. Le journal ne ment pas.
```

Ce qui se calcule au lieu de se stocker

```
CREATE VIEW effectif_courant AS
SELECT a.id AS arrivage_id,
       a.effectif_initial - COALESCE(SUM(m.nombre), 0)
       AS vivantes
FROM arrivages a
LEFT JOIN mouvements m ON m.arrivage_id = a.id
GROUP BY a.id, a.effectif_initial;

-- Valeur d'une bête, en centimes, recalculée
-- à chaque nouvelle dépense.
(a.effectif_initial * a.prix_unitaire_cts
 + COALESCE(SUM(d.montant_cts), 0))
/ NULLIF(e.vivantes, 0)
```

L'intégralité du schéma — les huit tables, les index, les quatre vues et les migrations — se trouve dans le dépôt : <https://github.com/Yedikhg/e-book1>

LA RÈGLE À RETENIR

Ne stockez jamais un chiffre que vous pouvez recalculer. Une colonne qu'on met à jour est une information qu'on efface.

Avant de passer au chapitre 6

- **Vos tables d'événements**, sans aucune colonne modifiable.
- **La liste de ce qui se calcule** : tout chiffre absent de vos tables et présent sur vos écrans.
- **Vos montants en entiers**, dans la plus petite unité de votre monnaie.
- **Les MANQUE non tranchés**, écrits dans le dépôt et non dans votre tête.

Le chapitre 6 écrit enfin du code. Vous verrez qu'après ces cinq chapitres, il en reste beaucoup moins à écrire qu'on ne le croit.

Le back-end, brique par brique

L'IA écrit très vite du code qui marche. Toute la difficulté est de l'empêcher d'écrire tout d'un coup.

Cinq chapitres sans une ligne de code. On y arrive enfin — et le premier réflexe est le mauvais : demander l'API complète.

Elle vous la donnera. Huit cents lignes, propres, compilables, livrées en une minute.

Et vous ne les relirez pas. Personne ne relit huit cents lignes qu'il n'a pas écrites. Vous les accepterez parce qu'elles compilent, et vous découvrirez ce qu'elles font quand quelque chose ira mal.

Une brique, une question

La discipline tient en une phrase : ne demandez jamais plus de code que vous ne pouvez en relire d'une traite.

Concrètement, une brique correspond à une des questions écrites au chapitre 2. Pas à une table, pas à une ressource : à une question.

Cette différence est tout le chapitre. Une table produit quatre opérations mécaniques — créer, lire, modifier, supprimer — dont trois n'ont aucun sens dans votre métier. Une question produit une opération réelle.

Personne, dans un élevage, ne « met à jour un mouvement ». On enregistre des morts, on vend, on encaisse, on corrige une erreur. Ce sont ces verbes-là qui doivent apparaître dans votre code.

Les écritures d'abord, les lectures ensuite

L'ordre de construction n'est pas indifférent.

Commencez par tout ce qui écrit ; gardez les lectures pour la fin.

La raison est simple : une lecture fausse s'aperçoit et se corrige. Une écriture fausse laisse des données abîmées que plus aucun correctif ne rattrapera.

Et comme le chapitre 5 a fait de vos lectures des vues calculées, elles ne coûtent presque rien à écrire une fois les écritures justes.

Concrètement : l'arrivage, puis la mortalité, puis la vente, puis le paiement, puis la correction. L'écran d'accueil vient en dernier, alors qu'il est le premier que verra l'utilisateur.

L'IA à cette étape · 6.1 — L'API

Le prompt de départ donne le schéma complet et le langage. Il est précis. C'est celui qu'on écrit quand on est pressé d'avancer.

CE QUE J'AI DEMANDÉ AU DÉBUT

Voici mon schéma PostgreSQL : [les huit tables du chapitre 5].

Génère-moi l'API REST complète en Go pour ce schéma, avec les handlers, la connexion à la base et la gestion des erreurs.

Code propre et idiomatique.

CE QUE J'AI OBTENU

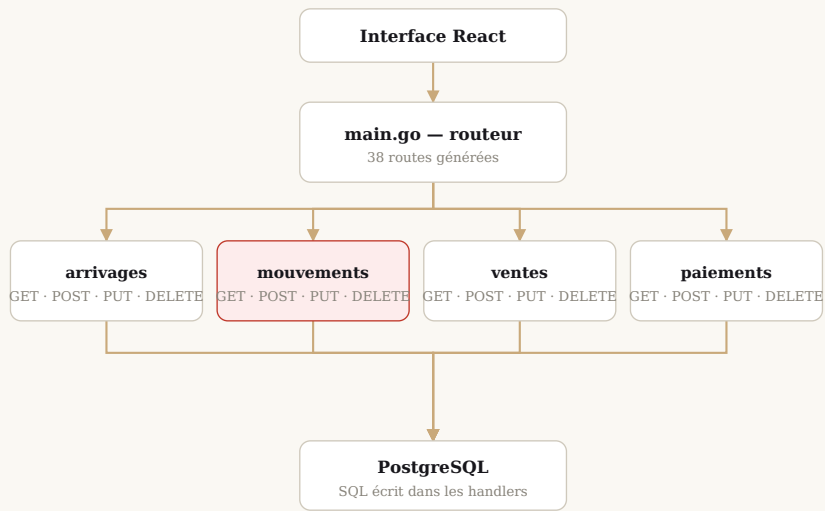
Huit cent quarante lignes, en un seul bloc. Trente-huit routes.

Pour chacune des huit tables : **GET** la liste, **GET** par identifiant, **POST**, **PUT**, **DELETE**. Le SQL est écrit directement dans les handlers. Les erreurs sont renvoyées en JSON avec le bon code HTTP.

Le code compile. Il est correctement formaté. Les noms sont bons. Un lecteur pressé n'y trouverait rien à redire.

Et c'est le problème : il n'y a rien à redire sur la forme.

L'ARCHITECTURE COMPLÈTE, TELLE QUE PROPOSÉE



Et quatre autres ressources identiques : parts, dépenses, clients, utilisateurs.

En rouge, la ressource qui n'aurait jamais dû exister sous cette forme.

POURQUOI CE N'ÉTAIT PAS BON

- **PUT et DELETE sur les mouvements.** Le chapitre 5 avait posé qu'un mouvement ne se modifie jamais. Trente-six heures plus tard, l'API l'autorisait. L'IA n'avait pas oublié la règle : je ne la lui avais pas donnée.
- **Aucune transaction.** Lire l'effectif puis écrire une mortalité en deux requêtes séparées : deux personnes qui saisissent en même temps peuvent tuer plus de bêtes qu'il n'en reste.
- **Du CRUD, pas des opérations.** Aucune des six questions du chapitre 2 n'a de route dédiée. Il faudrait les recomposer côté interface.
- **Huit cent quarante lignes d'un coup.** Je n'aurais rien relu. C'est le vrai défaut : les trois précédents sont invisibles à ce volume.

LE PROMPT QUI FONCTIONNE – LES SIX BRIQUES

[1] Tu es développeur Go. API d'une application de suivi d'élevage, un seul mainteneur, cinq utilisateurs.

[2] Schéma : [les huit tables]. Un mouvement est immuable ; une erreur se corrige par un mouvement d'annulation. Les montants sont des entiers de centimes. Une vente peut être encaissée en plusieurs tranches.

[3] N'écris qu'une seule opération : enregistrer une mortalité. Aucun autre handler. Interdiction d'exposer PUT ou DELETE. Aucun SQL dans le handler. Lecture et écriture dans la même transaction.

[4] Cas à couvrir : arrivage de 782 bêtes vivantes, saisie de 3 morts par l'utilisateur 4. Puis : saisie de 900 morts, qui doit échouer.

[5] Suis Effective Go et Go Code Review Comments pour le nommage et le traitement des erreurs.

[6] Trois fichiers séparés : http, service, store. Rien d'autre.

CE QUE J'AI OBTENU CETTE FOIS

Cent dix lignes. Trois fichiers. Une seule route.

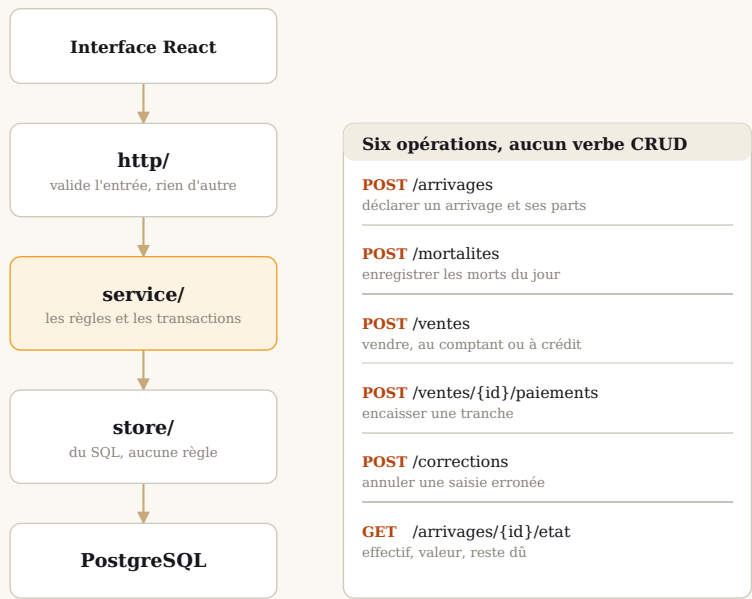
Le handler ne fait que lire le JSON et appeler le service. Le service ouvre une transaction, vérifie l'effectif, refuse si le nombre dépasse, écrit le mouvement. Le store ne contient que du SQL.

Le cas d'échec est traité : 900 morts sur 782 bêtes renvoie une erreur métier nommée, pas un code 500.

MANQUE : comment corriger une saisie erronée ? La contrainte *CHECK (nombre > 0)* du chapitre 5 interdit un mouvement négatif.

Ce dernier point m'a coûté une migration. J'y reviens dans deux pages.

L'ARCHITECTURE COMPLÈTE, APRÈS CORRECTION



Trois couches, six opérations. Les règles vivent au milieu, jamais aux extrémités.

La brique complète

Voici le service. Tout ce qui compte est dans la transaction : lire puis écrire séparément, c'est laisser deux utilisateurs se marcher dessus.

```
func (s *Service) EnregistrerMortalite(
    ctx context.Context, arrivageID int64,
    nombre int, par int64,
) error {
    if nombre <= 0 {
        return ErrNombreInvalide
    }
    return s.db.Tx(ctx, func(tx *sql.Tx) error {
        // L'arrivage existe-t-il ?
        exists, err := s.store.ArrivageExiste(ctx, tx, arrivageID)
        if err != nil { return err }
        if !exists { return ErrArrivageInconnu }

        vivantes, err := s.store.EffectifCourant(ctx, tx,
            arrivageID)
        if err != nil { return err }
        if nombre > vivantes {
            return ErrPlusDeMortsQueDeVivantes
        }
        return s.store.AjouterMouvement(ctx, tx, Mouvement{
            ArrivageID: arrivageID,
            Type:       MouvementMortalite,
            SurvenuLe:   s.horloge.Aujourd'hui(),
            Nombre:      nombre,
            SaisiPar:    par, // vérifié par la FK en base
        })
    })
}
```

Quand le code casse le schéma

La correction d'une saisie était impossible. Un mouvement négatif ? Interdit par la contrainte du chapitre 5. Un UPDATE ? Interdit par la règle du chapitre 5.

Ma belle table immuable ne prévoyait pas l'erreur humaine, qui est pourtant la chose la plus prévisible du monde.

```
-- Migration 004 : rendre l'annulation possible
ALTER TABLE mouvements
  ADD COLUMN annule_id BIGINT REFERENCES mouvements(id);

-- Un mouvement annulé et son annulation
-- sortent TOUS LES DEUX du calcul.
-- Condition 1 : exclure les mouvements annulés
--   (ceux qu'un autre mouvement référence)
-- Condition 2 : exclure les annulations elles-mêmes
--   (ceux qui ont annule_id rempli)
CREATE OR REPLACE VIEW effectif_courant AS
SELECT a.id AS arrivage_id,
       a.effectif_initial
       - COALESCE(SUM(m.nombre), 0) AS vivantes
FROM arrivages a
LEFT JOIN mouvements m ON m.arrivage_id = a.id
  AND m.annule_id IS NULL           -- pas une annulation
  AND NOT EXISTS (                 -- pas annulé
    SELECT 1 FROM mouvements c
    WHERE c.annule_id = m.id)
GROUP BY a.id, a.effectif_initial;
```

La première condition (*annule_id IS NULL*) exclut les mouvements d'annulation eux-mêmes — ceux qui portent une référence vers le mouvement qu'ils

LA RÈGLE À RETENIR

Demandez une brique à la fois, et bannissez les verbes CRUD. Le jour où l'IA vous rend huit cents lignes, ce n'est pas un gain de temps : c'est une dette que vous n'avez pas lue.

Avant de passer au chapitre 7

- **Vos opérations nommées en verbes du métier**, jamais en verbes HTTP.
- **Chaque écriture dans une transaction**, lecture de contrôle comprise.
- **Une erreur métier nommée** par règle refusée, pas un code 500.
- **La liste des endroits où le code a contredit votre schéma**. Il y en aura. Ce sont les plus instructifs.

Le chapitre 7 est celui que j'ai écrit en premier, et c'est le seul du livre dont je suis certain qu'il n'existe nulle part ailleurs.

La logique métier que l'IA rate

L'IA connaît tous les algorithmes du monde. Elle ne connaît pas le vôtre, parce qu'il n'est écrit nulle part.

Voici le chapitre le plus important du livre.

Tout ce qui précède — trouver le problème, écrire le cahier des charges, modéliser, construire brique par brique — converge ici. C'est le seul endroit où votre valeur ne peut pas être remplacée par un meilleur prompt.

Parce que la logique métier de votre application n'existe dans aucune donnée d'entraînement. Elle n'existe que dans la tête d'une personne, et c'est vous qui êtes allé la chercher.

Le calcul qui semblait trivial

Combien vaut un poulet aujourd'hui ?

La question paraît simple. On prend le coût total du lot, on divise par le nombre de bêtes vivantes, et voilà. N'importe quelle IA écrit cette division sans hésiter.

Et cette division est fausse. Pas approximative : fausse. Elle donne un chiffre qui a l'air juste, que personne ne conteste pendant des semaines, et qui fait perdre de l'argent à quelqu'un à chaque règlement.

Ce chapitre raconte pourquoi. Et pourquoi aucune IA ne pouvait le deviner à ma place.

Ce que la division simple ignore

Reprenons le cas réel. Mille bêtes, réparties neuf cents et cent entre deux propriétaires. Coût total à ce jour : trois mille dollars. Deux cents bêtes sont mortes.

La division simple dit : $3000 \div 800$, soit 3,75 \$ la bête. Élégant, et faux.

Parce que les dépenses n'arrivent pas toutes le premier jour. Le sac de nourriture acheté en semaine cinq n'a jamais nourri les deux cents bêtes déjà mortes en semaine deux. Les faire payer rétroactivement pour une nourriture qu'elles n'ont pas mangée fausse la valeur de toutes les autres.

Le vrai coût d'une bête vivante dépend du moment où chaque dépense est arrivée, et du nombre de bêtes vivantes à ce moment-là. Ce n'est pas une division. C'est une somme de tranches.

Pourquoi l'IA ne pouvait pas le savoir

Ce n'est pas une lacune de l'IA. C'est une propriété de la connaissance.

La règle « une dépense ne se répartit que sur les bêtes vivantes au moment où elle survient » n'est écrite dans aucun manuel de comptabilité, aucun cours, aucun dépôt de code public. Elle vit dans la tête de ma mère, qui ne l'a jamais formulée ainsi — elle le fait, sans le dire.

Une IA restitue ce qui a été écrit quelque part. Cette règle n'a jamais été écrite avant cette page.

LA RÈGLE À RETENIR

L'IA ne connaît pas votre métier, parce que votre métier n'est pas dans ses données. Votre travail n'est pas de le lui demander : c'est de le lui apprendre, une règle à la fois.

L'IA à cette étape · 7.1 — Le piège

Cette fois, le prompt naïf est excellent. C'est ce qui rend ce chapitre si important : on ne peut pas s'en sortir par un meilleur prompt.

CE QUE J'AI DEMANDÉ AU DÉBUT

Écris une fonction Go qui calcule la valeur courante d'une bête : coût total du lot divisé par le nombre de bêtes vivantes. Les montants sont en centimes.

CE QUE J'AI OBTENU

```
func ValeurParBete(coutTotalCts int64, vivantes int) int64 {  
    if vivantes == 0 {  
        return 0  
    }  
    return coutTotalCts / int64(vivantes)  
}
```

Cette fonction est parfaite. Elle est juste, sûre, bien nommée, elle gère la division par zéro. Et elle calcule la mauvaise chose.

POURQUOI CE N'ÉTAIT PAS BON

Le code est irréprochable. La formule est fausse. Aucune relecture de code, aucun test unitaire écrit par l'IA n'aurait attrapé ça — parce que le code fait exactement ce que j'ai demandé.

L'erreur n'est pas dans le code. Elle est dans la question. Et une IA ne corrige pas vos questions.

C'est là que le développeur cesse d'être un tapeur de prompts. Il faut savoir que le calcul est faux, et ça, aucun outil ne vous le dira. Il faut être allé dans le bâtiment, avoir vu les dépenses arriver à des dates différentes, et avoir compris que la mort d'une bête en semaine deux change le coût de toutes les autres.

C'est votre seul avantage réel sur la machine. Ce livre entier n'existe que pour vous amener à cette page.

LE PROMPT QUI FONCTIONNE – LES SIX BRIQUES

[1] Tu es développeur Go. Fonction de calcul comptable pour un élevage.

[2] La règle, que tu ne peux pas deviner : une dépense ne se répartit que sur les bêtes vivantes au moment où elle survient. Une bête morte avant une dépense n'en porte aucune part. Le coût d'une bête vivante est donc la somme, sur chaque période entre deux événements, de la dépense de la période divisée par les bêtes vivantes pendant cette période.

[3] Entiers de centimes uniquement. Aucun flottant. La somme des parts doit égaler le total dépensé, au centime près.

[4] Cas à couvrir : 1000 bêtes, 1000 \$ à l'achat. Semaine 2 : 200 morts. Semaine 5 : 2000 \$ de nourriture. Les 2000 \$ se répartissent sur 800 bêtes, jamais sur 1000.

[5] Suis Effective Go. Écris la fonction et la table de cas de test.

[6] Rends le code, puis explique en une phrase où un flottant aurait trahi.

CE QUE J'AI OBTENU CETTE FOIS

```
-- coûts par période, bêtes vivantes par période
func CoutParBete(periodes []Periode) (int64, error) {
    var total int64
    for _, p := range periodes {
        if p.Vivantes == 0 {
            continue
        }
        -- la part de CETTE dépense sur les bêtes
        -- vivantes à CE moment, en centimes
        total += p.DepenseCts / int64(p.Vivantes)
    }
    return total, nil
}
```

Reprenons l'exemple du chapitre 2 avec les deux méthodes.

Données : 1000 bêtes à 1 \$ (achat = 1000 \$).
Semaine 2 : 200 morts. Semaine 5 : 2000 \$ de nourriture.

Division simple (chapitre 2)

Total dépensé = 1000 + 2000 = 3000 \$

Vivantes = 800

Valeur = 3000 ÷ 800 = **3,75 \$ par bête**

Calcul par tranches (chapitre 7)

Période 1 (achat → semaine 2) : 1000 \$ ÷ 1000 bêtes = **1,00 \$**

Période 2 (semaine 5) : 2000 \$ ÷ 800 bêtes = **2,50 \$**

Le reste des centimes

La division entière laisse des restes. 200 000 centimes répartis sur 786 bêtes donnent 254 centimes par bête, avec un reste de 356 centimes.

Le piège : distribuer ces restes entre des bêtes individuelles. Le chapitre 5 a établi que les bêtes ne sont pas individuelles — ce sont des parts de propriétaires. Le reste doit donc être distribué entre propriétaires, au prorata de leurs parts.

```
-- Répartition du reste au prorata des parts.
-- Chaque propriétaire reçoit floor(reste * sa_part / total).
-- Les centimes restants vont au plus grand propriétaire.
func DistribuerReste(reste int64, parts []Part) {
    total := int64(0)
    for _, p := range parts { total += int64(p.Effectif) }
    distribue := int64(0)
    for i := range parts {
        part := reste * int64(parts[i].Effectif) / total
        parts[i].MontantCts += part
        distribue += part
    }
    // Le dernier centime va au plus grand propriétaire.
    // Toujours le même, toujours documenté.
    parts[0].MontantCts += reste - distribue
}
```

Règle choisie et assumée : le dernier centime va au propriétaire qui détient la plus grande part. C'est toujours le même qui « gagne », et c'est écrit noir sur blanc. Un arrondi invisible créerait un conflit ; un

Comment reconnaître une règle que l'IA va rater

Vous n'aurez pas toujours un calcul aussi net. Voici les signes qui annoncent une règle que l'IA ne peut pas deviner.

- **Personne ne sait l'énoncer, mais tout le monde l'applique.** Quand la personne dit « ça, on le fait comme ça, c'est tout », vous tenez une règle invisible.
- **Elle dépend du moment.** Toute règle où l'ordre des événements change le résultat échappe à un calcul naïf.
- **Elle décide qui gagne ou qui perd.** Répartition, arrondi, priorité de paiement : dès qu'il y a un lésé, il y a une règle métier.
- **Elle est propre à un lieu.** Une coutume locale, une façon de faire de famille, un usage de marché. Rien de tout cela n'est dans les données d'entraînement.

CE QU'IL FAUT RETENIR DE TOUT LE LIVRE

Le prompt qui fonctionne n'est pas celui qui est bien tourné. C'est celui qui contient une vérité que vous êtes seul à connaître. Les bons prompts vieillissent ; cette compétence-là, non.

Avant de passer au chapitre 8

- **Vos règles métier écrites**, chacune avec l'exemple chiffré qui la prouve.
- **Pour chacune, la version fausse** que produit le calcul naïf. C'est elle qui montre pourquoi la vôtre a de la valeur.
- **Vos règles d'arrondi**, écrites et identiques partout où de l'argent se divise.

La partie III est finie. Vous avez un back-end qui calcule juste. Le chapitre 8 le rend visible : c'est là qu'on découvre qu'un écran n'est pas un dessin, mais cinq états.

Le front-end : un écran, c'est cinq états

Un écran n'est pas un dessin. C'est cinq écrans qui portent le même nom.

Voici l'erreur qui coûte le plus cher côté interface, et l'IA la commet toujours : elle dessine l'écran plein.

L'écran plein, c'est celui de la maquette : des données bien rangées, un chiffre à chaque place. Il n'existe qu'un instant dans la vie réelle. Le reste du temps, l'écran est vide, en train de charger, en erreur, ou hors ligne.

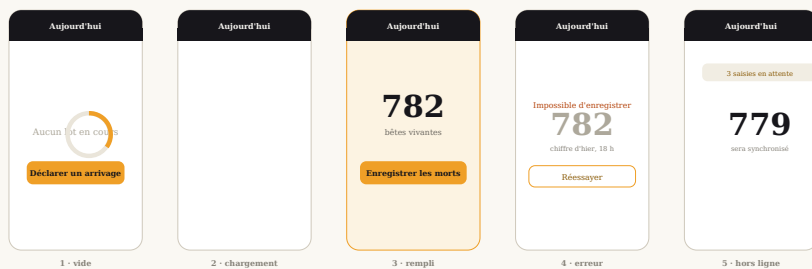
Un écran que vous n'avez dessiné qu'une fois est un écran dont vous avez oublié quatre versions sur cinq.

Les cinq états, nommés

Retenez ces cinq mots. Chaque écran de votre application les possède tous les cinq, que vous les ayez prévus ou non.

- 1 **Vide.** Aucune donnée encore. Le premier matin, le premier lot. C'est souvent le seul écran que l'utilisateur voit pendant une semaine, et celui qu'on oublie toujours.
- 2 **Chargement.** La donnée arrive. Sur une connexion qui coupe, cet état dure, et il doit rassurer sans mentir.
- 3 **Rempli.** Le seul que dessine la maquette.
- 4 **Erreur.** Quelque chose a échoué. L'écran doit dire quoi faire, jamais afficher un code technique.
- 5 **Hors ligne.** Cas particulier chez vous, cas central. L'écran montre la dernière donnée connue et sa date, et accepte la saisie quand même.

UN SEUL ÉCRAN, SES CINQ ÉTATS



« Aujourd'hui », cinq fois. La maquette ne montrait que le troisième.

L'IA à cette étape · 8.1 — Le composant

CE QUE J'AI DEMANDÉ AU DÉBUT

Écris le composant React de l'écran d'accueil qui affiche le nombre de bêtes vivantes et un bouton pour enregistrer les morts.

CE QUE J'AI OBTENU

```
function Accueil() {
  const [data, setData] = useState(null)

  useEffect(() => {
    fetch('/api/etat')
      .then(res => res.json())
      .then(setData)
  }, [])

  if (!data) return <p>Chargement...</p>

  return (
    <div>
      <h1>{data.vivantes} bêtes</h1>
      <button>Enregistrer les morts</button>
    </div>
  )
}
```

Ce composant ne plante pas, mais il gère un seul état sur cinq. « Chargement... » sera un mur blanc indéfini si le réseau coupe. Hors ligne, il reste bloqué. En erreur, il reste bloqué. S'il n'y a encore aucun lot, il affiche *null bêtes*. Et quand tout va bien, il refait la requête réseau à chaque ouverture, même si la donnée n'a pas changé.

LE PROMPT QUI FONCTIONNE

[1] Développeur React. Écran d'accueil d'une application utilisée hors ligne sur téléphone lent.

[2] La donnée vient d'un cache local d'abord, du réseau ensuite. Elle peut manquer, tarder, échouer, ou être périmée.

[3] Traite explicitement les cinq états : vide, chargement, rempli, erreur, hors ligne. Aucun ne doit faire planter le composant. Aucun message technique.

[4] Hors ligne : afficher la dernière valeur connue et sa date, accepter la saisie.

[5] Suis les règles des hooks React et les recommandations d'accessibilité pour les cibles tactiles.

[6] Un seul composant, les cinq états visibles dans le code.

Le code obtenu fait quatre-vingts lignes au lieu de dix. Les soixante-dix lignes de différence sont exactement les quatre états que la maquette ne montrait pas.

Le composant complet et ses états : <https://github.com/Yedikhg/e-book1>

LA RÈGLE À RETENIR

Un écran, c'est cinq états. Demandez-les nommément, sinon l'IA ne vous livrera que le plus rare : celui où tout va bien.

Avant de passer au chapitre 9

- **Chaque écran décliné en cinq états**, dessinés avant d'être codés.
- **L'état hors ligne traité comme normal**, pas comme une exception.
- **Aucun message d'erreur technique** visible par l'utilisateur.

Le chapitre 9 touche à l'argent. C'est le seul endroit du livre où une erreur ne se voit pas à l'écran : elle se voit sur le compte de quelqu'un.

L'argent : suivre n'est pas encaisser

Une erreur d'affichage se corrige. Une erreur d'argent se rembourse.

Jusqu'ici, une erreur se voyait à l'écran. À partir de ce chapitre, elle se voit sur le compte de quelqu'un — et elle ne se voit pas tde suite.

L'argent, dans une application, se divise en deux problèmes que tout le monde confond : le **suivre**, et l'**encaisser**. Les confondre est l'erreur qui coûte le plus cher de tout le livre.

Suivre l'argent, c'est de l'arithmétique : qui doit quoi à qui. Encaisser, c'est faire entrer de l'argent réel par un moyen de paiement. Le premier, vous le maîtrisez. Le second, ne le codez jamais vous-même.

La règle des entiers

Vous l'avez lue trois fois déjà. La voici entière, parce que c'est ici qu'elle se paie.

Un ordinateur ne sait pas représenter 0,10 exactement en flottant. Additionnez des dixièmes assez longtemps, et vous obtenez 0,999999. Sur un affichage, personne ne le voit. Sur un règlement de fin de lot où l'on répartit entre propriétaires, le total ne tombe plus juste, et quelqu'un accuse quelqu'un.

LA RÈGLE À RETENIR

Ne stockez jamais d'argent en flottant. Tout en entiers, dans la plus petite unité : le centime. Vous ne divisez qu'au dernier moment, et vous distribuez les restes.

Cette règle se décide avant la première table, jamais après. C'était la facture n° 3 du chapitre 3.

La source de vérité n'est pas votre application

Chez vous, l'argent entre par mobile money : M-Pesa, Airtel Money, Orange Money. C'est là que se joue le piège le plus commun.

La tentation est d'ajouter un bouton « payé » dans l'application, que l'utilisateur coche quand l'argent arrive. Ce bouton est un mensonge en puissance : il dit que l'argent est là, sans preuve.

La vérité d'un paiement ne vient jamais de l'écran. Elle vient de l'opérateur, par une notification serveur signée que vous vérifiez. Tant que cette confirmation n'est pas arrivée, le paiement est « en attente », pas « reçu ».

Deux règles de sécurité à ne jamais contourner :

- **Vérifiez la signature.** Chaque notification serveur est signée (HMAC ou certificat). Si vous ne la vérifiez pas, n'importe qui peut envoyer un faux paiement à votre application.
- **Vérifiez le montant.** Une notification peut arriver avec un montant différent de ce que vous attendiez. Comparez toujours avec le montant de votre vente avant de marquer « reçu ».

C'est aussi pour cela qu'on ne code pas l'encaissement soi-même. Un prestataire de paiement fait ce travail,

L'IA à cette étape · 9.1 — Le paiement

CE QUE J'AI DEMANDÉ AU DÉBUT

Ajoute une fonction pour enregistrer qu'une vente à crédit a été payée.

CE QUE J'AI OBTENU

```
func MarquerPayee(venteID int64) error {  
    return db.Exec(  
        "UPDATE ventes SET paye = true WHERE id = $1",  
        venteID)  
}
```

Trois erreurs en trois lignes : un booléen là où il faut un montant, un UPDATE là où il faut un journal, et aucune trace de qui a payé quoi, quand, par quel moyen.

LE PROMPT QUI FONCTIONNE – LES SIX BRIQUES

[1] Développeur Go. Encaissement d'une vente à crédit.

[2] Une vente peut être réglée en plusieurs tranches. Le reste dû est le total moins la somme des paiements. Un paiement provient de mobile money et n'est « reçu » qu'après confirmation signée de l'opérateur ; avant, il est « en attente ».

[3] Aucun booléen « payé ». Un paiement est une ligne, jamais un UPDATE. Montants en centimes. On refuse un paiement supérieur au reste dû.

[4] Cas : vente 240 \$, paiement de 120, puis de 120. Puis un paiement de 10 refusé car le solde est nul.

[5] Traite le paiement comme un patron Ledger : écritures immuables, solde calculé.

[6] La fonction, plus la table des cas de test.

La fonction, le webhook de confirmation et les tests : <https://github.com/Yedikhg/e-book1>

LA RÈGLE À RETENIR

Suivre l'argent est votre travail ; l'encaisser est celui d'un opérateur. La confirmation du paiement vient toujours du serveur de l'opérateur, jamais d'un bouton dans votre application.

Avant de passer au chapitre 10

- **Tous vos montants en entiers**, restes distribués, somme vérifiée.
- **Vos paiements en journal**, une ligne par tranche, aucun booléen.
- **La confirmation d'encaissement venant du serveur de l'opérateur**, vérifiée avant de marquer « reçu ».

La partie III est finie. L'application calcule juste et encaisse honnêtement. La partie IV la sort de votre machine — et c'est un autre métier.

Tester ce que l'IA a écrit

Si tous les tests générés passent du premier coup, ils ne testent rien.

Demandez des tests à une IA, elle vous en écrira des dizaines. Ils passeront tous. Et ils ne prouveront rien.

Parce qu'une IA qui écrit à la fois le code et ses tests teste ce que le code fait, pas ce qu'il devrait faire. Si le code se trompe, le test se trompe avec lui, dans le même sens. Les deux sont d'accord, et les deux ont tort.

Un test n'a de valeur que s'il pouvait échouer. Un test qui ne peut que passer est une décoration.

Les tests qui valent quelque chose

Vous n'écrirez pas mille tests. Vous en écrirez quelques-uns, aux bons endroits : là où le livre a montré que l'IA se trompe.

- **Les règles métier du chapitre 7.** Le coût par tranches, le prorata, les arrondis. C'est vous qui donnez le résultat attendu, calculé à la main. L'IA n'a pas le droit de le fournir.
 - **Les refus.** 900 morts sur 782 bêtes doit échouer. Un paiement supérieur au solde doit échouer. Un test qui vérifie qu'une chose interdite est bien refusée vaut dix tests du cas normal.
 - **Les restes.** La somme des parts égale-t-elle le total, au centime ? C'est le test qui attrape le flottant.
-

Écrivez le résultat attendu avant de lancer le test. Si vous le lisez après, vous validerez ce que le code a produit, pas ce qui est juste.

L'IA à cette étape · 10.1 — Les tests

CE QUE J'AI DEMANDÉ AU DÉBUT

Écris les tests unitaires pour ma fonction de calcul de valeur par bête.

CE QUE J'AI OBTENU — ET POURQUOI C'ÉTAIT UN PIÈGE

Douze tests. Tous verts. Et tous fondés sur le résultat que la fonction produisait déjà — y compris quand elle utilisait la division simple, qui était fausse.

L'IA avait pris le comportement du code pour la vérité. Elle testait la fonction contre elle-même.

C'est le piège le plus dangereux du livre, parce que le tableau de bord vert donne un sentiment de sécurité totale sur un calcul faux.

LE PROMPT QUI FONCTIONNE

[1] Développeur Go, tests de table.

[2] Ne lis pas ma fonction. Je te donne les cas et leurs résultats, calculés à la main. Ton travail est d'écrire les tests, pas de deviner les attendus.

[3] Inclus au moins trois cas qui doivent échouer : nombre négatif, dépassement d'effectif, paiement au-delà du solde.

[4] Cas : [ma table de vérité, écrite à la main]. Dont un cas où le total ne tombe pas juste, pour vérifier la distribution des restes.

[5] Suis les conventions de test de la documentation Go.

[6] Un fichier de test, un cas par ligne du tableau.

Les tests écrits ainsi ont trouvé deux erreurs — dont une dans un cas que je croyais juste. C'est le signe qu'ils testaient vraiment quelque chose.

La suite de tests complète : <https://github.com/Yedikhg/e-book1>

LA RÈGLE À RETENIR

Ne laissez jamais l'IA fournir le résultat attendu d'un test. Les cas et leurs réponses viennent de vous ; l'IA n'écrit que la mécanique autour.

Avant de passer au chapitre 11

- **Une table de vérité écrite à la main** pour chaque règle métier.
- **Un test de refus** par règle qui interdit quelque chose.
- **Le test des restes**, qui vérifie que rien ne disparaît au centime.

Le chapitre 11 met l'application en ligne. C'est là qu'on découvre la différence entre « ça marche chez moi » et « ça marche ».

Déployer et survivre à la production

Mettre en ligne, c'est facile. Rester en ligne, c'est le métier.

Le jour du déploiement est grisant, et trompeur. L'application est en ligne, elle répond, tout va bien. Le vrai travail commence le lendemain, quand quelque chose casse et que personne ne regarde.

Ce chapitre est court, parce que la règle est simple : préparez ce qui vous permettra de dormir, pas ce qui impressionne au lancement.

Les trois choses qui comptent vraiment

- **Les sauvegardes, testées.** Une sauvegarde qu'on n'a jamais restaurée n'est pas une sauvegarde, c'est un espoir. Restaurez-en une le jour un, avant d'avoir des données réelles à perdre.
- **Savoir quand ça casse avant l'utilisateur.** Une alerte simple qui vous prévient quand l'application ne répond plus vaut tous les tableaux de bord. L'utilisatrice ne doit jamais être votre système d'alerte.
- **Pouvoir revenir en arrière.** Le prochain déploiement cassera quelque chose. Ce qui compte n'est pas de l'éviter, c'est de pouvoir revenir à la version d'avant en une minute.

Remarquez ce qui n'est pas dans cette liste : la mise à l'échelle, les conteneurs, l'automatisation complète. Cinq utilisateurs n'en ont aucun besoin. Vous l'ajouterez le jour où le problème existera, et ce jour-là vous saurez pourquoi.

L'IA à cette étape · 11.1 — Le déploiement

CE QUE J'AI DEMANDÉ AU DÉBUT

Comment déployer mon application Go et PostgreSQL en production ?

CE QUE J'AI OBTENU

Une architecture pour mille fois mes besoins : conteneurs orchestrés, répartiteur de charge, base répliquée, chaîne d'intégration continue complète, supervision multi-services.

Impeccable pour une entreprise. Ingérable pour un seul développeur qui maintient le soir, et coûteux pour cinq utilisateurs.

Là encore, l'IA a répondu la moyenne d'internet : ce qu'on déploie dans les grandes entreprises, pas ce qui convient à un élevage.

LE PROMPT QUI FONCTIONNE

- [1]** Un seul développeur, cinq utilisateurs, budget minimal, maintenance le soir.
- [2]** Application Go compilée en un binaire, PostgreSQL, quelques centaines d'écritures par jour.
- [3]** Pas de conteneurs, pas d'orchestration, pas de réplication. La solution la plus simple qui tienne debout.
- [4]** Donne-moi : sauvegarde quotidienne automatique testée, alerte si l'application ne répond plus, procédure de retour arrière en une commande.
- [5]** Suis les douze facteurs applicables à ce contexte, ignore les autres.
- [6]** Une liste d'étapes numérotées, exécutables sur un petit serveur.

Les scripts complets : <https://github.com/Yedikhg/e-book1>

Le script de sauvegarde

Voici le strict minimum. Il tient en sept lignes et tourne tous les jours via cron.

```
#!/bin/bash
# sauvegarde.sh – à exécuter chaque jour à 3 h
DATE=$(date +%Y-%m-%d)
FICHER="/sauvegardes/elevage-DATE.sql.gz"

pg_dump elevage | gzip > "$FICHER"

# Garder les 30 dernières, supprimer le reste
ls -t /sauvegardes/*.gz | tail -n +31 | xargs rm -f

# Tester : restaurer dans une base jetable
# createdb test_restore
# zcat "$FICHER" | psql test_restore
# dropdb test_restore
```

Les trois lignes commentées en bas sont les plus importantes. Lancez-les au moins une fois, le jour de la mise en ligne, **avant** d'avoir des données réelles à perdre. Une sauvegarde qu'on n'a jamais restaurée est un fichier, pas une sauvegarde.

Le retour arrière

```
# Revenir à la version précédente en une commande
ln -sf /versions/elevage-precedent /app/elevage
systemctl restart elevage
```

Deux lignes. Le binaire précédent est toujours là, sous

LA RÈGLE À RETENIR

Déployez pour vos utilisateurs réels, pas pour un million imaginaire. La sauvegarde testée, l'alerte et le retour arrière valent mieux que toute l'architecture qu'on vous vendra.

Avant de passer au chapitre 12

- **Une sauvegarde déjà restaurée une fois**, pas seulement configurée.
- **Une alerte qui vous prévient avant l'utilisateur.**
- **Un retour arrière en une commande**, essayé au moins une fois.

L'application est en ligne. Il reste la marche la plus haute, celle que presque personne ne franchit : la transformer en premier client payant.

De l'application au client payant

Une application qui marche ne vaut rien tant que personne ne paie pour l'avoir.

Vous avez livré. Une personne réelle utilise votre application tous les jours. C'est déjà plus que la plupart des développeurs qui cherchent un contrat.

Mais il reste une marche, et c'est la plus haute : passer de « j'ai construit quelque chose » à « quelqu'un me paie pour le construire ». Ce chapitre ne parle plus de code. Il parle de preuve.

Votre application est une démonstration, pas un produit

Ne cherchez pas à vendre l'application de l'élevage à d'autres éleveurs. C'est le réflexe naturel, et c'est presque toujours une impasse : un produit se vend, se maintient, se supporte — un autre métier encore.

Ce que vous vendez, c'est la **preuve** que vous savez faire. Vous n'êtes plus quelqu'un qui a suivi des tutoriels. Vous êtes quelqu'un qui a pris le problème confus d'une personne réelle et l'a transformé en logiciel qu'elle utilise.

Cette phrase-là vaut de l'argent. Aucun clone de tutoriel ne peut la dire.

L'étude de cas qui vend

Votre meilleur argument commercial, vous venez de l'écrire sans le savoir : c'est ce livre, en plus court.

Racontez une histoire en trois temps, celle que vous connaissez maintenant par cœur :

- **Le problème réel.** Le cahier perdu, l'argent mal compté. Concret, humain, chiffré.
- **La difficulté que personne d'autre n'aurait vue.** Les parts, le prorata, le coût par tranches. C'est ce qui prouve que vous ne récitez pas un tutoriel.
- **Le résultat mesurable.** Le temps gagné, l'erreur évitée, la confiance retrouvée au moment des comptes.

Un client ne vous paie pas pour du code. Il vous paie parce qu'il a reconnu son propre problème dans le vôtre.

Combien facturer

La question la plus effrayante : combien ça coûte ?
Voici une méthode qui fonctionne quand on n'a aucune référence.

- **Estimez le coût du problème.** Combien l'erreur, la perte de temps ou l'argent mal compté coûtent chaque mois à cette personne. Si c'est 100 \$ par mois, votre application vaut moins que ça — sinon il n'a aucune raison de changer.
- **Facturez au projet, pas à l'heure.** « Je construis votre application pour X \$ et je la mets en ligne » est compréhensible. « Mon taux horaire est de Y \$ » ne l'est pas — personne ne sait ce qu'une heure de développement représente.
- **Commencez bas, mais jamais gratuit.** Un premier projet à 200 \$ est une preuve payée. Un projet gratuit ne prouve rien — ni pour vous, ni pour le client.

Les cinq premières minutes

Vous n'avez pas besoin d'un pitch. Vous avez besoin d'une phrase et d'un téléphone.

La phrase : « Je fais des applications pour des commerces qui tiennent leurs comptes à la main. J'en ai fait une pour ma mère — je peux vous la montrer. »

Le téléphone : ouvrez l'application, montrez l'écran d'accueil, et taisez-vous. S'il dit « moi aussi j'aurais besoin de ça », vous avez un prospect. S'il ne dit rien, passez au suivant.

Cherchez dans un rayon de cinq minutes à pied. Les pharmacies, les quincailleries, les dépôts de boissons, les coopératives. Pas sur LinkedIn — dans la rue.

LA RÈGLE À RETENIR

Ne vendez pas votre application : vendez la preuve qu'elle représente. Le premier client achète votre capacité à recommencer chez lui ce que vous avez fait chez vous.

Là où ce livre vous laisse

Vous êtes parti d'un cahier d'écolier. Vous avez un logiciel en ligne, utilisé chaque jour, qui calcule juste et encaisse honnêtement. Et vous savez faire tout le chemin, pas seulement écrire du code.

Surtout, vous avez la seule compétence que les modèles ne périment pas : reconnaître qu'une réponse d'IA est insuffisante, et savoir pourquoi. Les prompts de ce livre vieilliront. Cette compétence, non.

Le prochain problème est déjà autour de vous. Vous savez maintenant à quoi il ressemble.

Allez le chercher.

À garder sous la main

Les pages qui suivent sont faites pour être consultées pendant que vous construisez, pas lues d'une traite. Détachez-les mentalement du récit : ce sont vos outils.

Elles reprennent, sous forme utilisable, ce que le livre a démontré chapitre après chapitre.

ANNEXE A

Les douze diagnostics

Pour chaque chapitre, la réponse d'IA qui semblait bonne, et ce qui n'allait pas. C'est le cœur de la compétence : reconnaître l'insuffisance.

- 1 **Des idées d'applications** — la moyenne d'internet, jamais votre problème réel.
- 2 **Un cahier des charges complet** — des fonctionnalités inventées, la vraie difficulté manquée.
- 3 **La meilleure pile** — ce qui est populaire, pas ce qui vous convient.
- 4 **Une belle interface** — la moyenne du beau, pas le geste du matin.
- 5 **Un schéma propre** — normalisé, et incapable de répondre à vos questions.
- 6 **L'API complète** — huit cents lignes de CRUD que vous ne relirez pas.

- 7 **Le calcul « simple »** — un code parfait pour une formule fausse.
- 8 **Le composant** — l'écran plein, quatre états oubliés sur cinq.
- 9 **« Marquer payé »** — un booléen là où il faut un journal.
- 10 **Les tests générés** — verts parce qu'ils testent le code contre lui-même.
- 11 **Le déploiement** — une architecture pour mille fois vos besoins.
- 12 **Vendre l'application** — alors que ce qui se vend, c'est la preuve.

Un fil unique traverse les douze : l'IA rend la moyenne de ce qui existe. Votre valeur est tout ce qui n'est pas dans cette moyenne — votre problème, votre métier, votre contexte.

ANNEXE B

La bibliothèque de prompts

Tous les prompts qui fonctionnent du livre, réunis. Ils sont datés dans votre esprit : ils vieilliront. Ce qui ne vieillit pas, c'est leur structure en six briques.

Ne les copiez pas tels quels. Remplacez ma matière par la vôtre — c'est elle, et elle seule, qui fait la différence entre un prompt qui rend la moyenne et un prompt qui rend votre application.

Les prompts complets, prêts à copier et à adapter, sont dans le dépôt, tenus à jour :

<https://github.com/Yedikhg/e-book1>

ANNEXE C

Les six briques d'un prompt

La fiche à garder ouverte. Un prompt à qui il manque une brique baisse en qualité ; deux, il rend du générique.

- 1 **Rôle et contexte.** Qui vous êtes, ce que vous construisez, pour qui.
- 2 **Matière réelle.** Vos notes, vos phrases exactes, vos chiffres. Jamais un résumé.
- 3 **Contraintes.** Ce qui est interdit. La plus souvent oubliée.
- 4 **Exemple chiffré.** Un cas complet, avec de vrais nombres et le résultat attendu.
- 5 **Référence.** L'ouvrage ou la documentation dont vous voulez la discipline.
- 6 **Format de sortie.** Ce que vous voulez recevoir, et comment.

Les briques 3 et 4 manquent presque toujours aux prompts ratés. Commencez vos vérifications par elles.

ANNEXE D

La checklist de livraison

Avant de dire qu'une application est livrée, ces lignes doivent toutes être vraies.

- Chaque écran gère ses cinq états, hors ligne compris.
- Aucun montant en flottant ; les restes sont distribués.
- Aucun booléen « payé » ; les paiements sont un journal.
- Chaque écriture est dans une transaction.
- Chaque règle métier a un test dont l'attendu vient de vous.
- Une sauvegarde a été restaurée pour de vrai.
- Une alerte vous prévient avant l'utilisateur.
- Le retour arrière a été essayé une fois.

ANNEXE E

Le modèle complet

Le schéma final — huit tables, quatre vues calculées — est reproduit page 72. La vue la plus demandée est celle de la répartition par part, parce que c'est la seule qui touche à l'argent entre propriétaires.

```
CREATE VIEW repartition_par_part AS
WITH vivantes AS (
  SELECT arrivage_id,
         effectif_initial - COALESCE(SUM(nombre), 0) AS n
  FROM arrivages
  LEFT JOIN mouvements USING (arrivage_id)
  WHERE annule_id IS NULL
        AND NOT EXISTS (SELECT 1 FROM mouvements c
                        WHERE c.annule_id = mouvements.id)
  GROUP BY arrivage_id, effectif_initial
)
SELECT p.arrivage_id, p.proprietaire_id,
       p.effectif_initial AS part_initiale,
       -- mortalité au prorata, arrondi vers le bas
       p.effectif_initial
       - FLOOR(p.effectif_initial::numeric
               / a.effectif_initial * v.n) AS morts_estimes,
       -- bêtes vivantes pour cette part
       FLOOR(p.effectif_initial::numeric
             / a.effectif_initial * v.n) AS vivantes_part
FROM parts p
JOIN arrivages a ON a.id = p.arrivage_id
JOIN vivantes v ON v.arrivage_id = a.id;
```

Le reliquat d'arrondi (une bête maximum) est attribué au propriétaire ayant la plus grande part initiale, comme décrit au chapitre 7. Le code Go gère ce

ANNEXE F

Les références

La brique 5 nomme une référence. Voici celles qui reviennent dans le livre, par étape. Elles améliorent la moyenne ; elles ne garantissent rien. Une IA à qui l'on cite un ouvrage peut encore écrire du code médiocre — la vérification reste votre travail.

- **Règles testables** — la syntaxe Given / When / Then, documentée par Gherkin.
- **Base de données** — la documentation PostgreSQL, et l'ouvrage libre *Use The Index, Luke!*
- **Code Go** — *Effective Go* et *Go Code Review Comments*.
- **Argent** — le patron *Ledger* (grand livre à écritures immuables).
- **Déploiement** — *The Twelve-Factor App*, pour les facteurs applicables à votre échelle.

ANNEXE G

Glossaire

- **Arrivage** — un lot physique de bêtes reçu à une date, partagé entre propriétaires.
- **Part** — la quote-part d'un propriétaire dans un arrivage.
- **Mouvement** — tout événement faisant sortir une bête : mortalité, vente, don, consommation.
- **Vue calculée** — un chiffre recalculé à la demande, jamais stocké.
- **Prorata** — répartition proportionnelle aux parts.
- **Entier de centimes** — un montant stocké dans la plus petite unité, sans virgule.
- **Journal** — une suite d'écritures qu'on n'efface jamais ; on corrige par une écriture inverse.
- **Les cinq états** — vide, chargement, rempli, erreur, hors ligne.
- **Les six briques** — rôle, matière, contraintes, exemple, référence, format.
- **Transaction** — opération de base de données où toutes les écritures réussissent ensemble ou échouent ensemble, sans état intermédiaire visible.
- **Gherkin** — format d'écriture de règles testables en Given / When / Then.

Ce livre a été écrit à Goma, en République démocratique du Congo, à partir d'un projet réel : l'application de suivi d'élevage d'une famille.

Tous les prompts qu'il contient ont réellement été employés. Les réponses d'IA reproduites ont été rejouées à l'identique : les prompts sont ceux d'origine, les réponses sont celles que ces prompts produisent.

Le code, le schéma complet et la bibliothèque de prompts sont tenus à jour dans le dépôt :

<https://github.com/Yedikhg/e-book1>

Yedidya Kahire Gandura

Première édition – 2026